

Reuse, but Verify: Certified Maintenance of LLM-Computed Table Cells under Prompt Edits

Arjun Lohan
Independent Researcher
United States
arjunlohan7@gmail.com
lohan@usc.edu

Abstract

AI-native tables let users define columns in natural language: each cell is computed by a language model from a versioned prompt over its row. When the prompt is edited, deployed systems face a choice: recompute every cell, which is expensive and rate-limited, or silently reuse stale ones, which incurs unbounded error. We present sIVM, a maintenance layer that certifies which cached cells may be reused after a prompt edit, bounding the false-reuse rate of each certified stratum by a user budget α with probability at least $1 - \delta$. sIVM freezes cached-value strata, samples adaptively on a doubling look schedule, and bounds each stratum’s flip rate with an exact finite-population confidence bound; a strict mode certifies at a deflated threshold to bound the error of the reused cells alone. We also establish an impossibility floor: with the prompt unedited, no stratum reliably certifies below its own rate of self-disagreement. On two public corpora (89,184 Stack Overflow developer profiles; 210,250 Djinni candidate CVs) with 28,000 labeled cells, one primary model, and a working table system, a formatting-only edit certifies reuse of 82.6% of cells on average at $\alpha=0.1$ and of 88.1% at $\alpha=0.2$ ($\delta=0.1$; 1,000 replications). On the full corpus at $\alpha=0.2$, 90 oracle calls certify reuse of 81,424 of 89,184 cells, in a live run 55 days after the column was built that also measured drift of the served model, as a higher flip rate against the same cache. Across the calibration grid, a certificate exceeding its budget is issued with probability at most 2.7% per stratum at the least favorable population, against a nominal 5%. At least one edit certifies on 6 of 6 further models from four providers, while which edits are benign varies by model. We release the system, the labels, and five versioned prompt-edit pairs under a source-available noncommercial license.

1 Introduction

Interactive artificial intelligence (AI) tables turn a natural-language prompt into a *materialized view*: a column in which a language model computes every cell from the prompt and that cell’s row. Each cell is one draw from the model’s output distribution (§2 gives the notation). The category is now a product surface rather than a research proposal. Google Sheets evaluates an AI() formula per row and offers AI columns in tables [19], Airtable computes AI fields per record [1], Notion autofills database properties with a model [39], and Snowflake Cortex applies AI_COMPLETE and AI_CLASSIFY to table columns in SQL [51]. Microsoft’s Excel COPILOT function shipped as a preview and was

withdrawn in September 2026, a measure of how quickly this surface is moving [38]. These views are edited constantly: users iterate on prompts the way they iterate on spreadsheet formulas. Every edit invalidates the cache by construction, and today’s systems invalidate it wholesale: an edit to an 89,184-row column recomputes all 89,184 cells, or the user silently keeps stale values with no bound on how wrong they are.

Recent semantic data systems bound the error of *one-shot* query execution against a reference model whose answers are taken as correct (cascades, semantic operators, approximate processing with guarantees). Verified semantic caches bound reuse under a *fixed* definition (§7 names both lines). Neither answers the maintenance question an interactive table poses: *after this specific edit $\pi \rightarrow \pi'$, which cached cells are still valid, and with what guarantee?*

1.1 Running example

One column from our evaluation recurs throughout. Over 89,184 Stack Overflow (SO) developer profiles, a Boolean AI column asks “*Is this person a data-platform specialist?*”, judged from each profile’s role, databases, platforms, and tools. Its owner edits the prompt to insert two paragraph breaks and lowercase one word, changing no content word (the *formatting-only* edit of §5; §6 lists every edit we study). Recomputing means computing all 89,184 cells again. Keeping every cached value silently carries the edit’s flip rate into the table: the share of cells a recompute would change, 6.35% on this pair. sIVM instead groups the cached cells by their cached value (the two *strata*, cached-TRUE and cached-FALSE), recomputes a small seeded sample from each, and reuses a stratum only when a statistical bound on its flip rate clears the user’s error budget α . The procedure errs with probability at most δ . On this column, at $\alpha=0.2$ and $\delta=0.1$, 90 fresh calls certify reuse of 81,424 of the 89,184 cells. A live run did so 55 days after the column was built, against a model that had drifted in the meantime, and the draws stored when the column was built lead to the same decision (§5.5). The cells it refuses are the cached-TRUE stratum, which flips at 37.1% under the same edit on the labeled evaluation rows. A single bound over all the cells would clear the budget and reuse those cells too (§5.3).

1.2 Why intuition fails here

Three measurements reframe the problem (§5). First, large language model (LLM) cells are stochastic even at temperature zero [4, 41]: the same prompt on the same row disagrees with itself 5.0% of the time on our Boolean workload and 9.5% at default temperature (§5 gives the measurement behind each figure). “Did the edit change this cell?” is therefore only meaningful relative to

a noise floor, the rate at which the model disagrees with itself under the unedited prompt. Second, edit direction is not monotone: under a strictly scope-*widening* edit, 45.7% of cached-TRUE cells flipped, refuting the folk lemma that widening preserves positives. Third, an edit’s surface class is not a safe predictor of its semantic impact. On the same column, a synonym paraphrase written to preserve meaning flipped 8.90% of cells where a formatting-only edit flipped 6.35%. On a candidate-profile column from a second corpus, a formatting-only edit flipped 12.30% against 12.45% for a deliberate criteria change, rates that are indistinguishable. Neither pair orders the way the edits’ labels suggest. The study’s five edits are author-written, one per class per column, so they show that class can fail to predict flip rate, not how often it does. The certifier below never consults the class, and §5 shows the same edit’s flip rate varying widely across models. Both comparisons hold the column fixed deliberately: on the two column types we study, the column’s own stability moves the flip rate more than the edit does.

Sound reuse therefore has to be *measured*: sample a few fresh cells, statistically bound the flip rate of the rest, and reuse only what clears the user’s error budget.

1.3 Contributions

- (1) **Problem formalization** (§2): certified cross-version reuse for stochastic prompt-defined views. Two estimands (the population quantities a guarantee is about) are well-defined under decode pinning. One is the realized false-reuse rate over the cells a certified stratum presents, which we certify by default. The other is the same rate over the reused cells alone, which the strict mode certifies at a correspondingly lower threshold. An impossibility floor follows: with the prompt unedited, a budget α well below a large stratum’s self-flip rate is rarely certifiable, and §2 bounds how rarely.
- (2) **The sIVM procedure** (*semantic incremental view maintenance*; §3): a frozen stratification, an adaptive sample, and a futility rule. The stratification is by cached value, optionally refined by how strongly the edit’s changed terms match each row. Sampling follows a doubling schedule of *looks* (points at which the sample is inspected). The futility rule gives up on a stratum whose sample already exceeds the budget. Certification is per-stratum with an exact finite-population confidence bound, and the procedure offers two guarantee targets (presented cells; strict reuse set). Deciding at several looks is *peeking*; the validity theorem survives it by splitting the failure probability δ across the looks. Any frozen stratifier affects power only, never validity, where power is the chance of certifying a stratum that does meet the budget. The bound is a pluggable component: the exact bound is proven for the without-replacement sampling the procedure performs, and four alternatives (two empirical-Bernstein bounds, the binomial Clopper–Pearson bound, and a betting confidence sequence) are measured inside the same procedure.
- (3) **A working implementation** (§4) inside an AI-table system, released under a source-available noncommercial license. The system has a per-cell runner, a content-hash

cache, a cost ledger, and certificates (the persisted records of what cleared the budget), which the user interface (UI) surfaces.

- (4) **An empirical anatomy** (§5) of one primary model on two real corpora with 28,000 labeled cells. It measures the noise floor, the savings frontier over the error budget, and minority-stratum volatility, along with how much of it majority voting at write time removes. It includes a stratifier ablation (refinement leaves validity intact but costs power on this benchmark) and a calibration study of how often the procedure issues a certificate that exceeds its budget, against the nominal δ . It checks the cross-row independence assumption. It also covers 6 further models from four providers. A re-certification 55 days after the column was built measured drift of the served model as a higher flip rate against the same cache, and certified both budgets again. A free-text pilot (§8) calibrates a judge’s error on hand labels and folds it into the budget.
- (5) **A reusable benchmark** (§6): five versioned prompt-edit pairs with per-cell flip labels over public corpora redistributable under the Open Database License (ODbL) and the MIT License, enabling like-for-like evaluation of future maintenance systems.

2 Problem

Terminology. Table 1 defines the terms and the symbols this article uses. Four terms carry most of the weight. A *stratum* is a group of cached cells fixed before any sampling (by default, the cells that share a cached value). A *look* is a point in the sampling schedule at which the sample drawn so far is inspected and a decision may be taken. A stratum’s *floor* is its self-flip rate, the rate at which the model disagrees with its own earlier answer when nothing has changed. A *certificate* is the persisted record that a stratum cleared the budget, with the evidence it cleared it on.

Views and cells. An AI column is a view $V = (\pi, \mathcal{M}, \theta)$: a prompt template π with a typed output contract, a model $\mathcal{M}$, and pinned decode parameters θ . For row r_i , the bound prompt $\pi(r_i)$ interpolates only the fields $\beta(\pi)$ that π references; the stored cell c_i is *one draw* from the output distribution $f_V(r_i)$. Even at temperature zero this distribution is not degenerate: providers are nondeterministic across calls, as prior measurements of hosted models document [4, 41], in part because the floating-point reduction order inside serving kernels changes with the co-batched load [21, 57]. Every quantity below is therefore defined over draw randomness.

The maintenance event. The user edits the definition, $V \rightarrow V' = (\pi', \mathcal{M}, \theta)$. For each cached cell define the flip indicator $F_i = \mathbf{1}[c_i \neq X'_i]$, where $X'_i \sim f_{V'}(r_i)$ is a fresh draw under the new definition and $\equiv$ is the column’s equivalence relation (exact match for the Boolean and select columns studied here). The indicator F_i is Bernoulli with unknown mean $p_i = \Pr[c_i \neq X'_i]$. That mean deliberately folds together the edit’s semantic effect and decode noise on both sides: it answers the operational question of whether recomputing cell i now would return a value different from the one shown.

Table 1: Terms and notation.

Term	Meaning
stratum	a group of cached cells fixed before any sampling; by default the cells that share a cached value
look	a point in the sampling schedule at which the sample so far is inspected and a decision may be taken; named by its sample size (look 45, look 90)
flip	a fresh draw under the edited prompt that disagrees with the cached value
floor	a stratum’s self-flip rate: how often a fresh draw under the <i>unedited</i> prompt disagrees with the cached value
certificate	the persisted record that a stratum cleared the budget, with its evidence
refused	said of a stratum that is not certified; its unsampled cells are recomputed
futility	stopping a stratum’s sampling because the sample’s own flip rate already exceeds the budget
oracle draw (call label)	one fresh model call under the edited prompt for one row a stored fresh draw, used as the outcome a recompute would give; a model draw, not a human judgment
savings	the share of a column’s cells kept under a certificate (neither sampled nor recomputed)
frontier	savings as a function of the budget
loose, tight budget	$\alpha=0.2$ and $\alpha=0.1$
estimand	the population quantity a guarantee is about
power	the chance that a stratum which meets the budget is certified
null	a stratum whose realized flip count exceeds the budget; a <i>planted</i> null is a simulated stratum generated at a flip rate above the budget
unsafe certificate	a certificate issued for a null

Symbol	Meaning
$V = (\pi, \mathcal{M}, \theta)$	a view: prompt template, model, pinned decode parameters; $V' = (\pi', \mathcal{M}, \theta)$ after the edit
r_i, c_i	row i and its cached cell, one draw from $f_V(r_i)$
X'_i, F_i, p_i	a fresh draw under V' ; the flip indicator $\mathbf{1}[c_i \neq X'_i]$; its mean the column’s equivalence relation (exact match here); the theorem needs only the binary indicator it yields
$\equiv$	
j, n_j, s_j, p_j	a stratum, its size, its self-flip rate (floor), its flip rate (the mean of p_i over $i \in j$)
$\Delta, \beta(\pi)$	the prompt delta (content terms added or removed); the row fields π interpolates
$\mathcal{S}, m_j$	the sampled (freshly recomputed) cells; the number of them in stratum j
$\mathcal{R}$	the reuse set: cached cells kept under a certificate
L_j, t, n_t	stratum j ’s look schedule, a look in it, and the cells drawn by that look; n_0 is the first look and ℓ the most looks allowed in the bound: the stratum’s size ($N = n_j$), the cells drawn at a look, and the flips among them
N, n, k	the upper confidence bound on stratum j ’s flip rate at look t ; the sample’s own flip rate
$u_{j,t}, \hat{p}_j$	error budget on the false-reuse rate; the strict mode’s deflated threshold; upper bound on the probability that any certificate is wrong
α, α', δ	number of strata; the per-stratum failure probability δ/K ; the per-look level $\delta_j/ L_j $
K, δ_j, δ'	stratum j ’s realized flip count; the flips observed in its sample
M_j, k_j	realized false-reuse of stratum j over the cells it presents
$\text{FR}_j^{\text{pres}}$	the same over its reused cells alone
$\text{FR}_j^{\text{reuse}}$	

Term (continued)	Meaning
replication	a rerun of the certifier on the same labels with another sampling seed; the <i>main seed</i> gives the one run reported per configuration
snapshot	the model as served under one identifier over one period; a change of the served model ends it
pinned	held fixed: the decode parameters during a certification, and the one configuration every experiment uses
Bonferroni split	dividing a failure probability among several tests so that their failure probabilities cannot add up past it

Estimand and guarantee. A maintenance policy partitions the cached cells into a sampled set $\mathcal{S}$ (freshly recomputed), a reuse set $\mathcal{R}$ (stale values kept), and a recompute set. Two error rates matter, and they differ. The *presented-cells* rate of a certified stratum j is

$$\text{FR}_j^{\text{pres}} = \frac{1}{n_j} \sum_{i \in j \cap \mathcal{S}} F_i :$$

of all n_j cells the user now sees from that stratum, the fraction that would flip if recomputed (sampled cells are fresh and contribute zero). That zero is a convention of the single-draw estimand, not a property of recomputed cells: a recomputed cell is itself one draw, and a further draw would disagree with it at about the stratum’s self-flip rate. A flip is any disagreement with one fresh draw, decode noise included, as the previous paragraph defines it. The estimand charges reuse for exactly what the user would see change on recomputation, which is the operational question. The impossibility floor below is the price of that choice: an excess-over-floor target, $p_j - s_j$, would certify the identity edit for free. We deliberately do not adopt that target, because a user who recomputes sees one draw and lives with it. The *reuse-set* rate

$$\text{FR}_j^{\text{reuse}} = \frac{1}{n_j - m_j} \sum_{i \in j \cap \mathcal{R}} F_i$$

normalizes by the $n_j - m_j$ reused cells alone and is never smaller. A certificate bounds the stratum’s whole flip count, and hence $\text{FR}_j^{\text{pres}}$,

but not $\text{FR}_j^{\text{reuse}}$: the remainder is adversely selected, because certification happens exactly when the sample looked clean. sIVM therefore certifies the presented-cells estimand by default, and offers a strict reuse-set mode that certifies at the lower (*deflated*) threshold $\alpha(n_j - m_j)/n_j$, which bounds $\text{FR}_j^{\text{reuse}}$ by conservation of flips. On the *headline pair*, the SO formatting-only edit of the running example, at $\alpha=0.2$ the two rates read 3.20% and 3.28% (Table 2). Theorem 3.1 states both guarantees for the realized rates of this run, not for their expectations. Table 2 reports both realized rates for the main results. All guarantees are per-stratum and therefore hold simultaneously for every union of certified strata, not only the aggregate (baseline B2 in §5.3, aggregate-only certification, shows why the aggregate alone is operationally misleading).

sIVM makes no decision about an individual cell. Whether one cached cell would change under the edited prompt can be learned only by recomputing it, which spends exactly the call that reuse would save. The cheapest per-cell signal, prompt similarity, predicts flips at chance across a definition change (baseline B1 in §5.3). The signals that do carry information, the cached value and how strongly the edit’s changed terms match the row (the interaction score of §3), separate groups of cells, not single cells, and enter as strata. The unit of decision is therefore the stratum: an unsampled cached cell is reused if and only if the upper confidence bound

on its stratum’s flip rate clears the budget at a look the schedule reaches. That is the correctness criterion for reuse, and it is a statement about recomputation, not about ground truth. A certificate asserts that recomputing the stratum now would change at most an α share of its cells. It does not assert that the cells are right.

Reading the budget. The guarantee is a statement about the procedure, over a run’s sampling and oracle draws. With probability at least $1 - \delta$ the run issues no wrong certificate. In the default mode a wrong certificate is one for a stratum in which more than an α fraction of the cells would flip if recomputed. In the strict mode it is one for a stratum in which more than an α fraction of the reused cells would. The guarantee is not the probability that a certificate in hand is right. A stratum whose flip rate sits just above the budget is certified rarely, and every certificate it does receive is wrong; Table 2 contains such a configuration (§5.2). Both α and δ are the user’s to set, and the right values are workload-specific. A list that a person keeps working through tolerates a looser α than a column that gates a decision or feeds a downstream aggregate, and the strict mode exists for the latter (§8). The realized error under a certificate is typically far below α (3.20% against $\alpha=0.2$ on the headline pair, Table 2), because the bound must cover the worst population compatible with the sample. The frontier of §5.2 shows what tightening α costs in certifiable savings, and the floor below shows where tightening it becomes impossible.

Assumptions.

Assumption 1 (Pinned snapshot). The model $\mathcal{M}$ and the decode parameters θ are fixed during certification. Oracle draws are distributed as $f_{V'}(r_i)$ throughout the certification window and are independent *across rows*. Provider-side batching that correlates concurrent responses [21, 57] would violate this; our runner issues independent requests, and §5.6 compares sequential with concurrent requests. The validity proof uses independence only through non-interference: a sampled cell’s draw must not depend on which other cells were sampled or in flight. Rows with identical bound content share one cell by design (§4), and hence one draw; that sharing is fixed by row content before sampling and leaves non-interference intact. Certificates are scoped to the snapshot the oracle draws came from; the cache may be older. A change of the served model ends the snapshot, and §5.5 (the September run) measures such a change (change-of-world maintenance is complementary work).

Assumption 2 (Frozen stratification). The stratifier is a deterministic function of the cache, the edit, and row content, fixed before any oracle call. Any such stratifier preserves validity; its quality affects only power (§5.7).

Assumption 3 (Uniform within-stratum sampling). Sampled rows are drawn by a seeded uniform shuffle within each stratum, without replacement, and each sampled row receives one fresh oracle draw, independent across rows (Assumption 1, which also covers rows that share a cell). The sampling itself is fixed by the seeded canonical shuffle of §4. The calibration study of §5.6 checks that the implemented test holds the level Theorem 3.1 states, on planted strata whose rates sit just above the budget and on the fixed population that is least favorable to it.

No monotonicity shortcuts. Folk reasoning suggests one-sided safety: a scope-widening edit should preserve TRUE cells, so cached TRUEs could be reused for free. Our measurements refute the unconditional form: under a widening edit, 45.7% of cached-TRUE cells flipped. The widened definition admits every row the old one did, so these flips are not the edit’s meaning at work. Edit direction therefore enters sIVM only as a stratification hint; any directional certificate must be *conditional* on a measured violation rate, never assumed.

The impossibility floor is per-stratum. Let s_j be stratum j ’s self-flip rate: the probability that a fresh draw of a cell under the *unedited* prompt disagrees with the cached draw, *among the cells whose cached draw put them in stratum j* . For the identity edit $\pi'=\pi$, p_j equals s_j by definition. The floor concerns certificates that bound a stratum’s realized flip count M_j , the count that charges reuse for every disagreement with one fresh draw. Both modes of sIVM issue such certificates. Take any procedure that certifies a stratum with $M_j > \alpha n_j$ with probability at most δ_j . For the identity edit at a budget $\alpha < s_j$ it certifies stratum j with probability at most

$$\delta_j + \Pr[M_j \leq \alpha n_j] \leq \delta_j + \exp(-2 n_j (s_j - \alpha)^2),$$

where the second step is Hoeffding’s inequality [23] for draws that are independent across cells. (If w_c rows share cell c , the exponent’s n_j becomes $n_j^2 / \sum_c w_c^2$.) Under the unedited prompt the certifiable budget is therefore floored by model stability, up to a term that vanishes as the stratum grows. For a budget just below the floor that term is not small, and there power, not the floor, is what limits certification. Three scoping notes apply. First, the floor limits bounds on the whole count. A rule that bounds only the presented-cells rate can go below it by sampling more, because sampled cells are recomputed; in the limit it samples the whole stratum and reuses nothing. Second, the conditioning is not incidental. Because membership is decided by one draw, a minority-value stratum is selected partly *for* having flipped that way, so s_j is a property of the (cache, stratifier) pair, not of the column. Building the cache by a majority vote over several draws at write time lowers it (§5.7). Third, the floor is a bound for the identity edit only. An edit that sharpens the output distribution, or shifts it toward a stratum’s cached value, can drive p_j below s_j , and the model study contains such strata (§5.4). For every other edit the floor is a reference point, not a bound.

The floors are unequal, and the column-level average describes neither stratum. We measure each floor as defined, the stored cached value against three fresh draws of the same prompt per row. On our Boolean column the cached-FALSE stratum has $s=2.6\%$ (95% row-cluster bootstrap [2.0, 3.1]) while the cached-TRUE stratum has $s=23.0\%$ ([18.6, 27.4]), against 4.5% pooled. Rows are bucketed by the stored cached value the certifier itself stratifies on, and the interval resamples rows, since draws from one row are dependent. The column-level 5.0% quoted elsewhere is the rate between pairs of fresh draws. Three otherwise separate observations follow from this single measurement. The FALSE stratum is the only one that certifies at all, and it is certifiable in principle down to budgets near its own floor. At $\alpha=0.05$ it certifies in 74.4% of replications, some only at the deepest look of the schedule, for 38.6% mean savings (Table 2). There, power

rather than the floor is what binds. The TRUE stratum is never certified at any budget we tried: its estimated floor is above the loosest of them, though the interval reaches below it. Its size sets the corpus-scale savings ceiling: with 7,715 of 89,184 cells cached TRUE, reuse cannot exceed 91.3%. The ceiling belongs to *this* cache. A cache built by majority vote at write time lowers the stratum’s floor but not its flip rate under the formatting edit (§5.7). And any method that reuses cells from the cached-TRUE stratum inherits that stratum’s flip rate no matter what aggregate target it was given, which is what we observe when we run one (§5.3, B2’). Column-level stability is the wrong unit; the certifier’s stratification recovers the right one, and a standing refusal on a stratum is a stability diagnosis of that slice of the column.

3 The sIVM procedure

Given the edit $\pi \rightarrow \pi'$ and cached cells for the affected rows (Figure 1):

- (1) **Diff.** Compute a word-level longest-common-subsequence (LCS) diff of the prompts; collect content terms of added/removed spans (the *delta*). The delta is recorded in the certificate for provenance and consumed only by the optional refinement in step 2; it never enters the reuse decision itself.
- (2) **Stratify (frozen).** Partition rows by cached value and freeze the partition before any oracle call (Assumption 2). With a binary proxy this step is the stratification of proxy-certified selection [26], transported from one-shot selection to maintenance. What changes is the guarantee it carries: a bound per stratum instead of one aggregate bound over the selected set (§5.3, B2’, isolates exactly that difference). Optionally refine each value stratum by an *interaction score*, a measure of how strongly the delta matches the row (lexical delta-term overlap, or the embedding cosine between the delta text and the row’s bound content), bucketed into tertiles. This refinement leaves validity intact (Assumption 2). It pays only when within-value flip risk is heterogeneous, and otherwise costs power through the δ/K split (§5.7).
- (3) **Schedule.** For stratum j of size n_j , fix a doubling look schedule $L_j = (45, 90, 180, \dots)$ capped at n_j , with at most six looks. Split the stratum’s failure probability $\delta_j = \delta/K$ evenly across looks. Every experiment here pins $n_0=45$, at most six looks, and $\delta=0.1$. Here K is the number of cached values: 2 on the Boolean columns and 4 on the select column. In the free-text pilot of §8 and Appendix E there are 8 category strata. A look on a large Boolean stratum is therefore tested at the per-look level $\delta' = \delta_j/|L_j| = 0.1/12$, and a look on a large select stratum at $0.1/24$. A small stratum has fewer looks and a correspondingly larger per-look level.
- (4) **Sample and test.** Order the stratum’s rows canonically, shuffle them once with the seed (Assumption 3), and at each look t reveal one fresh draw for each row of the prefix (Assumption 1). Compute the exact finite-population upper confidence bound $u_{j,t}$ of Eq. (1) on the stratum flip

Algorithm 1 sIVM certification of an edit $\pi \rightarrow \pi'$

Require: cached cells $\{(r_i, c_i)\}$; edit $\pi \rightarrow \pi'$; budget α ; failure probability δ ; first look n_0 ; at most ℓ looks; mode $\in \{\text{default}, \text{strict}\}$; Assumptions 1–3

Ensure: reuse set $\mathcal{R}$, recompute set, certificate record

- 1: $\Delta \leftarrow$ word-level LCS diff of π and π'
 - 2: strata $1, \dots, K \leftarrow$ partition of the cells by cached value, optionally refined by an interaction bucket of Δ ; freeze
 - 3: **for** each stratum j with n_j cells **do**
 - 4: order $_j \leftarrow$ seeded shuffle of the stratum’s rows in canonical order; $L_j \leftarrow (n_0, 2n_0, \dots)$ capped at n_j , at most ℓ looks; $\delta' \leftarrow \delta/(K|L_j|)$
 - 5: **for** each look $n \in L_j$ **do**
 - 6: draw X'_i for the first n rows of order $_j$ not yet drawn; $k \leftarrow \{i \leq n : c_i \neq X'_i\}$
 - 7: $u \leftarrow$ exact upper bound of Eq. (1) at (k, n, n_j, δ')
 - 8: $\alpha' \leftarrow \alpha$ in the default mode, else $\alpha(n_j - n)/n_j$
 - 9: **if** $n < n_j$ and $u \leq \alpha'$ **then**
 - 10: certify stratum j ; **break**
 - 11: **end if**
 - 12: **if** $k/n > \alpha$ **then**
 - 13: refuse stratum j (futility); **break**
 - 14: **end if**
 - 15: **end for**
 - 16: refuse stratum j if its schedule is exhausted
 - 17: **end for**
 - 18: $\mathcal{R} \leftarrow$ unsampled cells of certified strata; recompute set $\leftarrow$ unsampled cells of refused strata
 - 19: persist the certificate (budget, Δ , per-stratum evidence) and apply $\mathcal{R}$ additively
-

rate at level $\delta_j/|L_j|$; certify and stop if $u_{j,t} \leq \alpha$; stop for futility if the empirical rate alone exceeds α .

- (5) **Apply.** Reuse all unsampled rows of certified strata (recording certificate provenance per cell); recompute the rest. Sampled rows are already fresh.

Algorithm 1 states the procedure in full. What the test conditions on is fixed before the first oracle call: the stratification, the look schedule with its per-look levels, the seed, the bound, and the equivalence relation. The budget α and the mode enter where a look’s bound is compared with a threshold, and in the futility stop, which can only end sampling. The schedule’s last look is sized from α only for budgets near 0.01 or below, smaller than any used here. The event on which every scheduled look’s bound holds therefore does not depend on them, so several budgets, and both modes, can be read from one sample path at no cost in δ (§5.5 reads three budgets). Two results in this article were computed after their draws had been seen, and are marked as retrospective where they appear: the replay of the August draws under the exact bound (§5.5) and the graded relation of §5.2. The guarantee is per certification. Each certified edit spends its own δ , so over m certifications the chance that at least one certificate is wrong is at most $m\delta$.

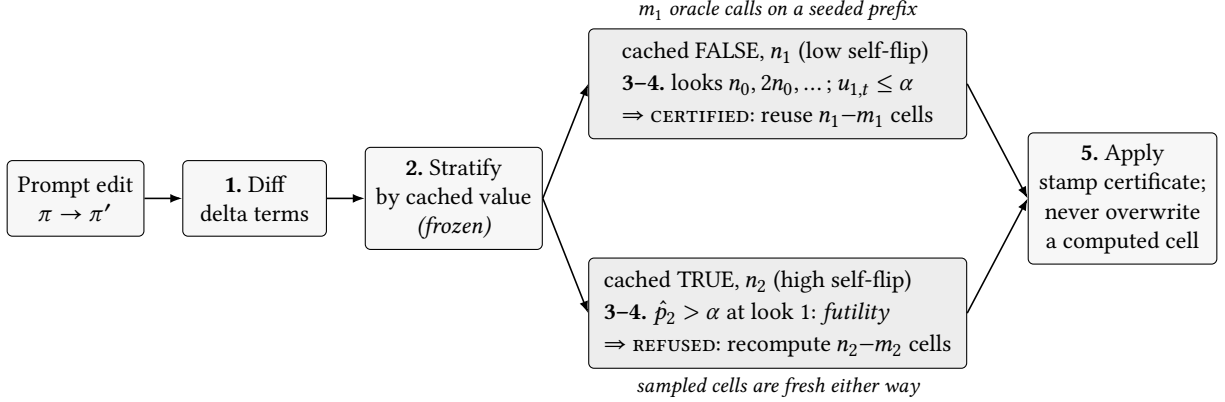


Figure 1: The procedure on the configuration our experiments pin. Stratification is frozen before any oracle call, so it moves power but never validity (Assumption 2); only steps 3–4 spend model calls, and they stop on evidence about a stratum’s rate, so their cost is bounded independently of the stratum’s size. The two lanes are the outcome on the primary model’s Boolean column: the stable stratum certifies and is reused under a certificate, while the volatile one is refused at every budget we tried and recomputed. A refusal is a stability diagnosis of that slice of the column (§2), reached at the cost of one look instead of a full recomputation.

The bound. Fix a stratum of N cells ($N = n_j$), and give every cell one hypothetical fresh draw under V' , the coupling on which the presented-cells estimand is defined. Let M be the number of cells whose fresh draw flips. The sample of n cells is drawn uniformly without replacement and independently of those draws, and shows k flips. Conditional on the N outcomes, k is Hypergeometric(N, M, n), and $\Pr[\text{Hyp}(N, M, n) \leq k]$ is non-increasing in M . The exact upper confidence bound at level δ' is

$$u = \frac{1}{N} \max\{M : \Pr[\text{Hyp}(N, M, n) \leq k] > \delta'\}, \quad (1)$$

the finite-population form of the Clopper–Pearson bound [13], to which it tends as N grows. Because the coverage holds conditionally on every population realization, it holds unconditionally over sampling and draw randomness together, and it bounds the realized flip count M rather than its expectation. This is the natural bound for a binary loss under the sampling the procedure performs. On a clean sample its cost is $\ln(1/\delta')/n$ to first order. The empirical-Bernstein bounds pay range terms with constants of $7/3$ (Maurer–Pontil, on $\ln(2/\delta')$) and $7/3 + 3/\sqrt{2}$ (Bardenet–Maillard, on $\ln(5/\delta')$) that a binary loss does not need. On a clean sample of $n=90$ from a stratum of 1,800 cells at the pinned per-look level, the exact bound reads 0.050. In the 81,469-cell deployment stratum it reads 0.052; there it coincides with the binomial limit to three digits. On the clean sample of 90 from 1,800 cells, the Maurer–Pontil bound [37] reads 0.144, and the Bardenet–Maillard without-replacement bound [6] reads 0.317. §5.7 and Appendix B measure the difference on every configuration and price a betting confidence sequence [56]; time-uniform boundaries [24] are discussed in §8 as the remaining lever. The system exposes the alternatives as ablation arms.

THEOREM 3.1 (VALIDITY UNDER PEEKING). *Let K be the number of strata produced by the frozen stratifier, $\delta_j = \delta/K$, and let every*

look use the exact bound of Eq. (1) at level $\delta_j/|L_j|$. Under Assumptions 1–3, with probability at least $1 - \delta$ over sampling and oracle draws, every certified stratum j has realized whole-stratum flip count $M_j \leq \alpha n_j$; consequently $\text{FR}_j^{\text{pres}} \leq \alpha$ for every certified stratum and for every union of certified strata (the empty certification is vacuously valid). In the strict (reuse-set) mode, where certification at sample size m_j requires $u_{j,t} \leq \alpha(n_j - m_j)/n_j$, the same argument gives $M_j \leq \alpha(n_j - m_j)$, hence $\text{FR}_j^{\text{reuse}} \leq \alpha$. The statements hold for any per-look bound that is a valid upper confidence bound at that level for uniform without-replacement draws, including the Bardenet–Maillard bound [6]; the Maurer–Pontil bound [37] is stated for independent draws and is retained only as an ablation arm.

PROOF. Fix stratum j and condition on its n_j fresh-draw outcomes, of which M_j flip. If $M_j > \alpha n_j$, certifying at look t requires $u_{j,t} \leq \alpha < M_j/n_j$, i.e., the observed count to fall in the lower tail that Eq. (1) rejects at level $\delta_j/|L_j|$; by the monotonicity of the hypergeometric tail in M , that event has probability at most $\delta_j/|L_j|$ over the sampling, for every population realization, hence unconditionally. Because the looks are prefixes of one shuffle fixed before sampling, the prefix of size n_t (the t -th entry of L_j) is, marginally, a uniform without-replacement sample of that size, whether or not the procedure reaches look t . The bound therefore fails at any one scheduled look with probability at most $\delta_j/|L_j|$, and a union over all scheduled looks covers whichever look the data stops at. That union bound over the $|L_j|$ looks and the K strata gives δ ; this is the Bonferroni instance of Learn then Test [2], each (stratum, look) pair testing the null $M_j > \alpha n_j$ with a certificate as the rejection. The futility rule only stops sampling, never certifies, so it cannot increase the error probability. On the good event, $\text{FR}_j^{\text{pres}} = (M_j - k_j)/n_j \leq \alpha$, where k_j is the number of flips observed in the sample, and any size-weighted union of certified strata inherits the bound. In the strict mode the good event is $M_j \leq \alpha(n_j - m_j)$;

at worst every flip falls in the remainder, so $\text{FR}_j^{\text{reuse}} \leq \alpha$. The conditioning uses Assumption 1 only through non-interference: the population of fresh-draw outcomes must be the same whichever cells the shuffle selects. $\square$

Remark. The Bardenet–Maillard bound is proven for the same uniform without-replacement draws (its Serfling factor shrinks the variance term as the sampled fraction grows; its constants are those of Theorem 4.3 of [6]), so the argument transfers to it unchanged. The Maurer–Pontil bound is stated for independent draws. Whether it remains valid at the sampled fractions our runs reach is not something we prove, which is why the pinned bound is the exact one. Those fractions are large: the schedule’s deepest look draws 1,440 cells of a stratum, most of a benchmark stratum, and the certifier samples up to 74% of a benchmark table.

Power. In a large stratum a clean sample certifies once $(1-\alpha)^n \leq \delta'$, i.e., at $n^* = \lceil \ln \delta' / \ln(1-\alpha) \rceil$. Under the pinned procedure on a Boolean column ($K=2$, six looks), this gives $n^*=94$ at $\alpha=0.05$, against 257 for the Maurer–Pontil bound. The doubling schedule cannot stop on n^* : the first look at or above it is the one it takes. A clean stratum of $n_j=5,000$ therefore pays 180 draws in calibration. A clean stratum of $n_j=600$ pays 90, because its finite-population correction and its shorter schedule’s larger per-look level bring the earlier look under α . Either way, savings require strata much larger than n^* , i.e., large tables, which is precisely the deployment regime. Near the frontier, certifying a stratum with true rate p requires roughly $n \gtrsim z_{\delta'}^2, p(1-p)/(\alpha-p)^2$ samples, where $z_{\delta'}$ is the standard normal quantile at level δ' (the normal approximation to the exact test). This explains the measured collapse of savings as $\alpha \downarrow p$. On SO formatting (Table 2), mean savings are 88.1%, 82.6%, and 38.6% at $\alpha=0.2, 0.1$, and 0.05 , with refusal at 0.02 .

4 Implementation

sIVM runs inside *lore*, a working AI-table system. Its natural-language search bar compiles to a versioned filter specification executed identically on MySQL and Elasticsearch (conformance-tested for count, page, sort, aggregation, and hydration parity on 89,184 rows). AI columns are per-cell jobs with typed structured outputs, bounded concurrency, retry, and a content-hash cache keyed on *only the bound fields* $\beta(\pi)$. Unrelated row changes therefore never invalidate the cache, and content-duplicate rows share one computation. Every run passes a pre-run cost gate and lands in a ledger; prompt edits create immutable versions (the sIVM substrate); certificates persist with full strata evidence, and reused cells carry visible provenance in the UI. Decode is pinned ($T=0$) for all cell computation per Assumption 1. The table’s certify action runs the pinned procedure of §3 in the default mode on the rows of the user’s current view, at most 2,000 per request. Each request is a certification of its own, with its own δ . The corpus-scale runs of §5.5 call the same certifier function from a script.

What the user sees. In the table view a reused cell carries a shield mark beside its value. Its tooltip shows the cell’s stored rationale with the note that it was reused from the previous version under a statistical certificate. A cache hit is annotated as one in the same place. Certifying a column version returns the certificate’s summary to the user: the rows in scope, α , the cells sampled, reused,

and to be recomputed, and the flips observed in the sample. A refused stratum has no mark of its own: its cells are recomputed and served as ordinary fresh cells, and the refusal is visible in the certificate record. Whether this surface changes how a person acts on the column was not studied (§8).

Storage model. Cells are keyed by (column, row, prompt version), so versions of a column *coexist* instead of replacing one another, and a secondary index on (column, version, model, content hash) serves the cache. This is the one design decision sIVM depends on. Systems that key a cell by row alone, or that hash the prompt into the cache key, destroy the old materialization at the moment of the edit. The question this article asks cannot then be posed: there is nothing left to certify. Keeping versions side by side turns “which cached cells still transfer?” into a join between two versions of one table, which is why the certifier is a query over the store, not a service beside it. Two details of that query matter for reproducibility and for Assumption 1. The certifier orders each stratum’s rows canonically (by content hash, then row identifier) before the seeded shuffle, so the sample path is a function of the seed and the data alone. An earlier version of the certifier took the order the storage engine happened to return. That order changed when the query planner switched indexes, and the same seed then drew a different sample (Appendix C). And a model snapshot is a version of its own: oracle draws taken after a change of the served model are stored under a new prompt version carrying the same template. Draws from two snapshots therefore never share a key and cannot be mixed in one certification path.

Certificates are records, not decisions. A certificate persists the budget α , the prompt delta, the per-stratum evidence (size, cells sampled, the bound at each look, certified or refused), and the counts of sampled, reused, and recomputed cells. Each stratum’s entry also names the level, the mode, and the bound it was tested under; that field is new in this release, and the certificates of §5.5 carry those settings in their run artifacts. A certificate is therefore auditable after the fact: the realized-error audits in §5.5 are recomputed, from the row identifiers each run persisted and the stored cells, by a script that imports nothing from the certifier. Applying a certificate is a single INSERT...SELECT that carries certified values forward under the new version and stamps the certificate identifier on each one.

That insert carries an anti-join against the target version, so reuse is strictly *additive*: a certificate can supply a value where none was computed, and can never overwrite one that was. An earlier implementation used an upsert, which is the obvious way to write it and is silently destructive. Applying a certificate after a partial recompute reverted freshly computed cells to the stale values the certificate had been asked to reuse. The bug is invisible in aggregate error, because a reverted cell and a reused cell are the same row with the same value. It surfaces only if something outside the certifier knows what that cell should have said. Any system that maintains stochastic cells across versions has this hazard, and the guard belongs in the write path, not in code review.

Three further rules govern live operation. A sampled cell whose oracle call fails is counted as a flip and left for recomputation. The rule is conservative: a failure can add to the flip count the bound

sees and never subtract from it, so it can cost a certificate and cannot create one. (The studies of §5 measure flip rates on completed draws; Appendix D reports the independence check under both conventions.) Reuse does not chain. A reused cell is a copy of the previous version’s value, not a draw of the new one, and the certifier takes only computed cells as its cache. At the next edit a reused cell is therefore recomputed, and the savings reported here are per edit (§8). Finally, the sampling seed defaults to a hash of the column and its two versions. Successive certifications of one column therefore draw different samples, and each sample is reproducible from the certificate record. The experiments of this article set the seed explicitly (§5.1).

5 Experiments

This section reports the setup (§5.1), the main results (§5.2), the baselines (§5.3), the other models (§5.4), the deployment-scale runs (§5.5), the validity checks (§5.6), and the ablations (§5.7). Appendices B to H hold the detail: the bound comparison, the deployment runs, the independence tests, the free-text pilot, the calibration study, the strict mode, and the baselines.

5.1 Setup

Workloads. All measurements use two public corpora and three prompt-defined columns (§6 gives the exact edits, and the Availability section the sources). On the 2023 SO Developer Survey (89,184 respondent rows) a Boolean column asks whether the respondent is a data-platform specialist, judged from structured facets of the row (role, databases, platforms, tools). The column exists twice, as the production column of the system’s own table and as a lab replica seeded from that column’s second version, so that edits can be studied without touching the first. (The system has no outside users; “production” names the column, not a user base.) On Djinni English candidate profiles (210,250 rows) a four-way ordinal select column assigns a seniority tier from the position, years of experience, and free-text profile. Five edit pairs across the three columns cover formatting-only edits, synonym rewording, scope widening, and a criteria change; each is labeled on a seeded $n=2,000$ evaluation vector of its corpus.

Scope. One budget-class reasoning model carries the full experimental program. A further 6 models from four other providers, three of them successive Gemini generations, replicate the lab column’s two pairs; 5 of them replicate all five pairs. The quantitative frontier reported below is specific to the (edit, model) pair, while the qualitative structure is what the model study tests (§8). Table 3 lists every model identifier and run date. The gateway exposes no snapshot identifier finer than these names (the primary model’s carries a date suffix; the others do not), so the run dates are the snapshot record. The September re-certification below shows that what one identifier serves can change between run dates.

Models and decoding. Every cell in the primary program uses DeepSeek V4 Flash, called through the Vercel AI Gateway (gateway identifier `deepseek/deepseek-v4-flash-0731`). Those calls were made between 4 August 2026 and 5 August 2026, and again on 29 September 2026 for the re-certification and the assumption checks (and on 30 September 2026 for the larger concurrent arm of

the independence check). The model study calls the further models through the same gateway from 15 August 2026 to 29 September 2026. Every call passes temperature 0 and no other decode parameter. Maximum output length and reasoning effort stay at the provider’s defaults. No decode seed is sent. The seeds of this article govern row sampling only: one main seed gives the single run reported per configuration, and 1,000 further seeds give the sampling replications. Cells are requested as typed structured outputs against a JSON schema through the gateway’s client library. A call whose output fails the schema or whose request errors is retried by the client. A cell that still fails is stored with an error status and excluded from labels and from every rate in this article. No such cell exists among the cells written through the column runner; the probe scripts that call the model directly report their own exclusions where they occur. The embedding proxy in baselines B1 and B3 and in the interaction stratifier is `openai/text-embedding-3-small`, also through the gateway. Every labeling pass, sample, and replay is seeded and reproducible from released scripts.

Labels and spend. Flip labels are single fresh draws under the new version, which matches the estimand: a label answers “would a recompute differ from the cache now?” Labels are therefore noisy themselves. A three-draw relabeling of the headline pair measures by how much. Depending on the instrument, that pair’s flip rate reads between 6.0% and 7.2%, and 4.8% of rows change label between two of the labelings. Table 4 reconciles each flip-rate figure quoted for the headline pair on the evaluation vector, and for its cached-TRUE stratum, with the edit, the cells, and the instrument behind it. It is population functionals, not labels, that the guarantee addresses. This is the same picture the LLM-as-annotator literature draws, where model labels reach crowd-worker or expert agreement on many tasks but with task-specific noise that has to be validated rather than assumed [18, 42, 53, 59]. Ledgered model spend across every experiment in this article is \$12.01 for 100,683 billed cells, read from the system’s own cost ledger. That figure is a lower bound. A further 146,753 cells were computed by probe scripts that call the model directly (the self-flip probes, the majority-vote study, the multi-draw relabeling, the model study, the independence check, and the free-text pilot’s draws).

5.2 Main results

Table 5 gives the strata the certifier decides on, and Table 2 the outcomes. On the Boolean columns the cached-FALSE stratum holds about nine cells in ten and flips at a few percent, while the cached-TRUE stratum flips at a third or more. On the select column the Junior tier flips least, and the other three tiers flip at rates between the two headline budgets. The outcomes follow from these strata. On the stable Boolean column the certifier reuses aggressively where it is safe. The formatting edit certifies in every replication at $\alpha=0.2$, with 88.1% mean savings, and in nearly every replication at $\alpha=0.1$, with 82.6%. On the main seed its realized presented-cells error is 3.20% at both budgets. At $\alpha=0.05$ it still certifies in 74.4% of replications, but for 38.6% mean savings, because much of the stratum is sampled before the bound clears. The synonym and scope-widening edits certify at $\alpha=0.1$ in most replications and are refused below it, apart from the few scope-widening certificates accounted for below. On the less stable select

Table 2: The pinned primary procedure (cached-value strata; exact bound; adaptive doubling looks $n_0=45$, six looks; $\delta=0.1$; presented-cells estimand) on all five edit pairs, $n=2,000$ each at temperature 0. Column *flip* is the pair’s label flip rate. Columns *oracle* and *reused* are the calls spent and the cells reused by the main seed. FR_{pres} and FR_{reuse} are the main seed’s realized false-reuse over presented cells and over the reuse set alone. Column *savings* is the mean over $B=1,000$ sampling replications on frozen labels, with a 95% percentile interval (PI). Column *cert.* is the share of those replications that issued any certificate. Column *exceed.* is the share of the certifying replications (not of B) whose realized reuse-set rate exceeded α , a metric the default mode does not certify (§2). The interval is taken over the replications, so a configuration that certifies in only a few of them shows a mean above an interval of zeros (scope widening at $\alpha=0.05$). The table is generated mechanically from the result files.

Edit	α	flip	oracle	reused	FR_{pres}	FR_{reuse}	savings % [95% PI]	cert.	exceed.
SO formatting-only	0.02	6.35%	225	0	–	–	0.0 [0.0,0.0]	0.0%	–
SO formatting-only	0.05	6.35%	1,485	374	0.55%	2.67%	38.6 [0.0,81.7]	74.4%	0.0%
SO formatting-only	0.10	6.35%	90	1,769	3.20%	3.28%	82.6 [72.7,88.5]	98.9%	0.0%
SO formatting-only	0.20	6.35%	90	1,769	3.20%	3.28%	88.1 [86.2,88.5]	100.0%	0.0%
SO synonym rewording	0.02	8.90%	90	0	–	–	0.0 [0.0,0.0]	0.0%	–
SO synonym rewording	0.05	8.90%	90	0	–	–	0.0 [0.0,0.0]	0.0%	–
SO synonym rewording	0.10	8.90%	90	0	–	–	55.6 [0.0,89.0]	81.4%	0.0%
SO synonym rewording	0.20	8.90%	225	1,645	5.81%	6.44%	87.4 [82.3,89.0]	99.9%	0.0%
SO scope widening	0.02	9.45%	90	0	–	–	0.0 [0.0,0.0]	0.0%	–
SO scope widening	0.05	9.45%	765	0	–	–	1.0 [0.0,0.0]	1.2%	100.0%
SO scope widening	0.10	9.45%	405	1,441	4.50%	5.62%	66.3 [0.0,87.8]	88.8%	0.0%
SO scope widening	0.20	9.45%	90	1,756	5.33%	5.47%	86.6 [85.6,87.8]	99.9%	0.0%
Djinni formatting-only	0.02	12.30%	180	0	–	–	0.0 [0.0,0.0]	0.0%	–
Djinni formatting-only	0.05	12.30%	315	0	–	–	0.0 [0.0,0.0]	0.0%	–
Djinni formatting-only	0.10	12.30%	495	383	4.17%	8.09%	8.4 [0.0,32.7]	73.8%	8.4%
Djinni formatting-only	0.20	12.30%	953	1,047	8.33%	10.12%	52.1 [32.7,73.6]	100.0%	0.0%
Djinni criteria change	0.02	12.45%	180	0	–	–	0.0 [0.0,0.0]	0.0%	–
Djinni criteria change	0.05	12.45%	315	0	–	–	0.0 [0.0,0.0]	0.0%	–
Djinni criteria change	0.10	12.45%	540	378	3.52%	6.88%	14.4 [0.0,32.4]	82.8%	5.4%
Djinni criteria change	0.20	12.45%	745	1,255	7.50%	11.00%	48.6 [32.4,69.5]	100.0%	0.0%

Table 3: Every model the article uses, by gateway identifier, with the gateway’s list prices as fetched on 29 September 2026.

Gateway identifier	Role	Run dates	\$/M tokens in / out
deepseek/deepseek-v4-flash-0731	primary model: every cell of the main program, the deployment run, the drift and independence checks, the free-text pilot’s cells and its judge	4 August 2026 to 5 August 2026; 29 September 2026; 30 September 2026	0.08 / 0.15
openai/text-embedding-3-small	embedding proxy (baselines B1, B3; interaction stratifier)	with the main program	0.02 / 0.00
google/gemini-2.5-flash-lite	model study, all five pairs	15 August 2026; 29 September 2026	0.10 / 0.40
google/gemini-3-flash	model study, lab column’s two pairs	15 August 2026	0.50 / 3.00
zai/glm-4.7-flash	model study, all five pairs	15 August 2026; 29 September 2026	0.07 / 0.40
alibaba/qwen3.7-flash	model study, all five pairs	15 August 2026; 29 September 2026	0.03 / 0.13
openai/gpt-5-nano	model study, all five pairs	29 September 2026	0.05 / 0.40
google/gemini-3.8-flash	model study, all five pairs	29 September 2026	0.75 / 3.75
meta/muse-spark-1.3-contributor	free-text pilot: second labeler of the calibration pairs, reported beside the hand labels	30 September 2026	0.10 / 0.20

column both edits certify at $\alpha=0.2$ in every replication, for about half the column, and at $\alpha=0.1$ in most replications, for less than a sixth of it. Of the 20 default-mode configurations, 12 certify in at least one replication. Budgets below a stratum’s flip rate are refused, and those refusals are the frontier of Figure 2. Realized presented-cells error is inside α in every main-seed certification.

The α -frontier and adaptive economics. Savings collapse as α approaches a stratum’s flip rate (Figure 2), as the power analysis predicts; below that rate the certifier refuses. The doubling schedule prices both directions. A stratum whose sample rate exceeds the budget is refused at that look, the first one for the strata farthest from the budget. Near-frontier certifications escalate sampling only while the data stays consistent with certifiability. A single refusal near the frontier is not evidence that the stratum exceeds the budget. The synonym edit at $\alpha=0.1$ is refused on the

main seed and certified in 81.4% of the sampling replications, a power miss the futility rule trades for its early stops.

Noise floors and column stability. Replicate self-runs give the two column types’ self-flip rates. The Boolean column’s floor is 5.0%, measured between pairs of fresh draws over the $n=2,000$ rows. The four-way select’s floor is 6.5% on a smaller probe of $n=200$, in which every observed self-flip was between adjacent tiers. At default temperature the Boolean floor reads 9.5% on an $n=200$ two-draw probe whose $T=0$ arm reads 3.0%. The default-temperature floor alone exceeds a 5% budget, which motivates decode pinning. A formatting-only edit flips somewhat more cells than no edit does: on the Boolean column, 7.2% against 5.0% with fresh draws on both sides. On these two columns the flip rate tracks the column’s own floor more closely than the edit’s class: the Djinni criteria change lands at 12.45%, indistinguishable from

Table 4: Reconciliation of the flip-rate figures quoted for the headline pair on the evaluation vector and for its cached-TRUE stratum. Each is a different quantity, a different instrument on the same quantity, or the same instrument on an earlier draw; none is a restatement of another.

Rate	Edit	Cells	Instrument (artifact)
<i>Headline pair: SO formatting-only, Boolean lab column</i>			
6.35%	formatting v1→v2	$n=2,000$ evaluation vector	stored v2 oracle cells against the stored v1 cache, one draw each side (127 flips); the Table 2 label instrument (exp8)
6.0%	formatting v1→v2	same rows	the same comparison on the v2 cells as first drawn (120 flips), recorded before 1,811 of them were re-drawn on 5 August 2026 (exp10)
6.30%	formatting v1→v2	same rows	majority of three draws on each side (126 flips; exp10)
7.2%	formatting v1→v2	same rows	fresh single draws on both sides, neither the cache (exp10)
5.0%	identity (no edit)	same rows	pairs of fresh draws of one version: the column-level self-flip rate (exp10)
2.6%, 23.0%	identity	cached-FALSE, cached-TRUE rows	the stratum floors: the stored v1 value against each of three fresh v1 draws (4.5% pooled; exp10)
<i>Cached-TRUE stratum figures</i>			
37.1%	formatting v1→v2	cached-TRUE rows of the vector	single fresh draw against the cache (exp4)
24.7%	formatting v1→v2	same	vote-of-3 on both sides (exp4)
34.4%	formatting v1→v2	same	vote-of-3 on both sides, recomputed from the three-draw labels of a separate run (exp10)
17.6%	identity	188 rows whose three-draw v1 majority is TRUE	the stored v1 value against that majority: the floor of a cache built by vote (exp10)
41.0%	formatting v1→v2	same	each fresh v2 draw against that majority (exp10)
45.7%	widening v1→v2, production column	199 cached-TRUE rows	single fresh draw against the cache (Wilson [39.0, 52.7])
35.3%	formatting v1→v2	cached-TRUE cells B2 reused at $\alpha=0.1$	realized error among the cells the aggregate-only baseline reused, an adversely selected subset, not a population rate (exp7)
37.2%	formatting v1→v2	same at $\alpha=0.2$	same (exp7)
44.6%	widening v1→v2, production column	cached-TRUE cells B2 reused at $\alpha=0.2$	same (exp7)

Table 5: The strata of the five edit pairs: the cells of each cached value on the $n=2,000$ evaluation vector, and how many of them flip under the edit.

Edit	cached value	cells	flips	flip rate
SO formatting-only	FALSE	1,814	58	3.2%
	TRUE	186	69	37.1%
SO synonym rewording	FALSE	1,825	118	6.5%
	TRUE	175	60	34.3%
SO scope widening	FALSE	1,801	98	5.4%
	TRUE	199	91	45.7%
Djinni formatting-only	Junior	743	54	7.3%
	Mid	559	91	16.3%
	Senior	529	73	13.8%
	Lead/Principal	169	28	16.6%
Djinni criteria change	Junior	738	46	6.2%
	Mid	574	87	15.2%
	Senior	528	89	16.9%
	Lead/Principal	160	27	16.9%

the formatting edit at 12.30%. Five author-written pairs cannot say how general that is.

Certificates that exceed the budget. A valid procedure may issue wrong certificates, in at most a δ share of runs. Table 2 contains one configuration where it does: SO scope widening at $\alpha=0.05$, whose certified stratum flips at a rate just above the budget. It certifies in 1.2% of replications. Every one of those certificates is unsafe in the sense of Theorem 3.1, and that share sits inside the per-stratum $\delta_j = \delta/K = 0.05$ of that column. This is an exact test spending part of its δ_j on a stratum just past the budget, the behavior the calibration study measures at scale (\$5.6). The reuse-set rate, which the default mode does not certify, exceeds α on some default-mode certificates in 3 configurations (the exceed. column of Table 2).

The strict mode. The strict mode guarantees the reused cells alone, at a threshold that shrinks as sampling deepens. That stronger guarantee is nearly free where a stratum’s flip rate is well below the budget and expensive near it. On the headline edit it costs nothing at $\alpha=0.2$ and two points of mean savings at $\alpha=0.1$ (80.6% against 82.6%). At $\alpha=0.05$ it certifies in 4.5% of replications, where the default mode certifies in 74.4%. On the select column it cuts mean savings by more than a third at $\alpha=0.2$ and certifies in at most 2.7% of replications at $\alpha=0.1$. Appendix G tabulates both modes at the three budgets where either certifies.

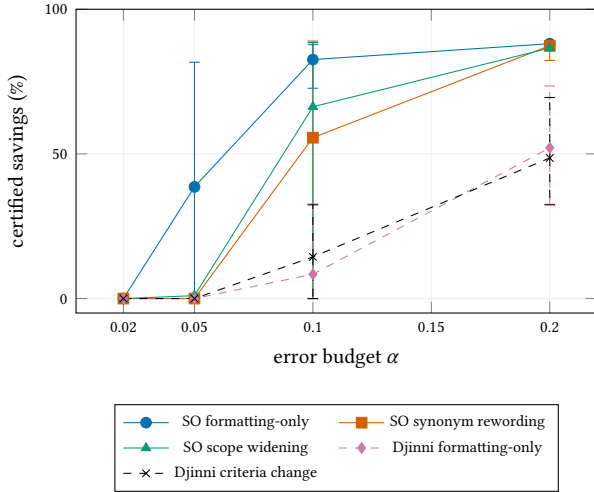


Figure 2: The α -savings frontier under the pinned procedure: the mean over 1,000 sampling replications per point, with the 95% percentile interval of Table 2 as the bar, generated from the result files. Savings fall away as α approaches the flip rates of the strata that certify.

A graded relation (retrospective). Exact match is the strictest relation a select column can use, and the relation is the user’s to state (§8). Every self-flip observed on the seniority column was between adjacent tiers. After the exact-match results were in, we therefore re-scored the stored labels of that column’s formatting edit under a tolerance: a cell flips only when the fresh tier is more than one step from the cached one. Under the tolerance the edit flips 4 of 2,000 cells, where exact match flips 12.30%. The pinned procedure then certifies it in every replication at $\alpha=0.05$, a budget that exact match refuses, for 66.3% mean savings. This is a weaker guarantee, not a tighter one. It bounds moves of two tiers or more and says nothing about moves of one tier. The certifier is unchanged: Theorem 3.1 needs only a fixed binary flip indicator, which need not come from a transitive relation. The tolerance would be the wrong relation for the column’s criteria edit, whose stated purpose is to move some candidates up one tier. Of that edit’s 249 exact-match flips, 244 are one-tier moves, and a tolerance would certify away the change the user asked for.

5.3 Baselines

Four comparisons isolate what sIVM adds. Table 6 lines the policies up, and Appendix H gives the detail of each.

(B1) *Verified-cache similarity has no cross-version signal.* The per-row similarity between the v1-bound and v2-bound prompts, the quantity a verified semantic cache thresholds, predicts cell flips at chance. The area under the receiver-operating-characteristic curve (AUC) is 0.454 on the formatting edit and 0.499 on the widening edit. Adding “a version key” to such a cache therefore cannot produce a sound cross-version policy. Our interaction score carries signal when the edit is semantic (AUC 0.669 on widening) and none when flips are noise-driven (0.406 on formatting). That is why the cached *value* carries the stratification in the formatting case.

(B2) *Aggregate-only certification*, the pinned procedure without strata, is valid for its aggregate estimand and silently corrupts subgroups. At $\alpha=0.2$ on the formatting edit it certifies 97.8% of the cells at 6.39% aggregate error, while the cached-TRUE cells it reuses carry 37.2% error. Per-stratum bounds are what make the guarantee mean something for every user-visible slice of the table.

(B2') *A SUPG-style competitor, matched.* The closest existing method is proxy-certified selection [26]: score rows with a cheap proxy, spend an oracle budget labeling a sample, and threshold the proxy so that the selected set’s error clears the target. We transported it to maintenance with the proxy the reduction in §3 names, the cached value itself. We matched it to sIVM on the oracle calls, the labels, the error target, the bound, and the total failure probability δ . Matched this way it wins on volume and loses on where its errors land. At $\alpha=0.2$ it reuses nearly every cell on both edits and satisfies the aggregate bound it was given. The extra volume is the volatile stratum: 183 cached-TRUE cells on the formatting edit, from a stratum whose floor is 23.0%. sIVM reuses none of them. An aggregate target is satisfied by any mixture that averages out, so it buys volume with the correctness of the least stable subgroup. The result does not depend on the choice of proxy, nor on the competitor’s looser per-test level (Appendix H).

(B3) *Guarantee-free embedding transfer* (GPTCache-style [5], $\tau=0.95$), the deployed status quo, reuses every cell on the formatting edit and 84.1% of them on the widening edit, absorbing error at about the population flip rate. Its threshold controls volume, never error. A sweep of τ takes the reused share from nearly everything to nothing within a window about 0.03 wide, while the error among reused cells stays at the population flip rate (Appendix H). The trivial policies, always recompute and always reuse, bracket the design space in Table 6. sIVM is the only row whose reused cells carry a per-stratum bound.

5.4 Other models

Every measurement above uses one model. We therefore ran the lab column’s two pairs on 6 further budget-class models from four other providers (Table 7), and the remaining three pairs on 5 of them (Table 8). The runs use the same seeded rows as the primary evaluation vector, the same prompts, $T=0$, and one fresh draw per cell. Each model’s cache is its own draw under the old version, and its floor a second draw of that version. A flip is therefore the model disagreeing with itself across the edit, never a comparison with the primary model. The two tables report the replication analysis of Table 2, repeated on each model’s stored draws.

What transfers. Every model has a measurable self-flip floor on the lab Boolean column, from 0.4% to 34.8% (Table 7). On the production column one model, Gemini 2.5 Flash-Lite, shows no self-flip in its rows (Table 8). Certified reuse is not a property of the primary model. On each of the 6 further models at least one edit certifies in at least 99.8% of replications at the loose budget. And unsafe certificates stay rare. Across the 54 cells of the two tables, a certificate for a null is issued in at most 1.5% of a cell’s replications, inside the per-stratum level of every column. A realized presented-cells error above the budget occurs in at most 0.7%.

Table 6: Reuse policies on the SO formatting and scope-widening edits at $\alpha=0.2$ ($n=2,000$, main seed). Realized error is the flip rate among reused cells; the trivial policies’ errors follow by construction (always-recompute has none; always-reuse carries the population flip rate). B2 additionally reports the error of its reused cached-TRUE subgroup in parentheses. B2’ is matched to the 90 labels sIVM spent; B2 runs the pinned schedule without strata and stopped after 45 labels on the formatting edit (360 at $\alpha=0.1$) and 90 on the widening edit.

Policy	Guarantee	Formatting		Widening	
		reused	error	reused	error
Always recompute	none needed (zero error)	0.0%	0.0%	0.0%	0.0%
Always reuse (status quo)	none	100%	6.35%	100%	9.45%
B3 GPTCache, $\tau=0.95$	none	100%	6.35%	84.1%	9.8%
B2 aggregate-only	aggregate	97.8%	6.39% (37.2%)	95.5%	9.37% (44.6%)
B2’ SUPG-style, matched	aggregate	95.5%	6.54%	95.5%	9.58%
sIVM (this work)	per stratum	88.5%	3.28%	87.8%	5.47%

What does not transfer. Which edits are benign does not transfer. The formatting-only edit is the benign one on the primary model. It certifies in every replication on Gemini 3 Flash and Gemini 3.8 Flash, and in almost none on Gemini 2.5 Flash-Lite and GPT-5 nano. On GLM-4.7-Flash it certifies only a small stratum (3.3% mean savings). On Gemini 2.5 Flash-Lite the edit flips 25.45% of cells against a floor of 4.85%. Decomposed, its two paragraph breaks alone flip 23.2% and its one-word case change alone 11.0%. The edit also moves the share of TRUE answers on that model, from 21.8% to 34.8%. On GLM-4.7-Flash it moves that share the other way, from 80.0% to 61.8%. Self-disagreement alone flips cells both ways and leaves the share where it was, so on these two models the edit shifts the model’s decision. On the three models where the formatting edit fails, the synonym rewording certifies a stratum in most replications at the loose budget instead, because its flips concentrate in one cached value and leave the other reusable. The ordering that the edits’ labels suggest (formatting benign, rewording not) therefore inverts on some models and holds on others. Benignness is a property of the (edit, model) pair, and the certifier measures it per stratum without consulting the label.

Floors are per stratum. The floor binds for the identity edit only (§2), and some strata of this study flip less under an edit than under none. The two clearest are on the lab column. Under the formatting edit GLM-4.7-Flash’s cached-FALSE stratum flips 17.0% against an identity floor of 24.4%, and GPT-5 nano’s cached-TRUE stratum flips 24.3% against 44.9%. GPT-5 nano’s pooled floor, 34.8%, sits above every budget in the grid. Its one certifying stratum on the lab column sits below that pooled figure for the same reason: a floor is a per-stratum quantity, measured under the first version’s prompt.

5.5 Deployment scale

To run the certifier at the scale of a full corpus, we materialized the Boolean lab column across all 89,184 rows. “Deployment scale” means that scale throughout: a script calls the certifier the table uses, and no outside users are involved. The materialization came to 78,956 billed cells in a single ledgered run costing \$9.31, which took 7.2 hours of wall clock at 32-way concurrency. The remaining 10,228 rows cost nothing, served by content-hash caching or

already materialized by earlier passes. We then certified the formatting edit against that column, with the real model as oracle, at three budgets ($\alpha=0.2$, 0.1, and 0.05) on two model snapshots over one seeded sample path (Table 9). The first certification uses the oracle draws of the live run of August 2026. The second was drawn live on 29 September 2026, 55 days later, against the same cache. The budgets share the path at no cost in δ : the per-look bounds do not depend on α , so the event on which they all hold covers each budget. The August run itself certified under the Maurer–Pontil bound, which Theorem 3.1 does not cover, and the system applied that certificate. Table 9 therefore replays the August draws under the exact bound. Those rows are retrospective: the bound was fixed after the draws had been seen. The September rows were drawn live under the exact bound, fixed beforehand. Appendix C gives the certificate the system applied in August.

The August draws, replayed. At the loose budget the exact bound certifies the cached-FALSE stratum at look 45. The whole certification therefore costs **90 oracle calls** and reuses **81,424 cells**, 91.3% of the column. The ceiling is 91.3%, set by the cached-TRUE stratum the certifier never certifies. The procedure thus captures essentially all the reuse available to capture, and the residue is a property of this cache (§2). The tight budget, $\alpha=0.1$, costs 225 calls. At $\alpha=0.05$ the stored draws run out before the schedule’s deepest look. Drawing the missing cells later would mix two snapshots, so the August evidence is silent there. Audited against the oracle-computed cells of that snapshot, the reused rows flip at 3.15% (89 of 2,821 audited rows). A separate verifier script recomputes every audit of Table 9 from the row identifiers each run persisted and the stored cells.

The September run: drift, measured. Assumption 1 scopes a certificate to the model snapshot its oracle draws came from, so we reran the certification live on 29 September 2026: same model identifier, same gateway, same prompts, same cache, same seeded path. The snapshot had moved. On the 720 cached-FALSE rows the August run had sampled, the edit’s flip rate against the cache rose from 3.5% to 6.9% (McNemar $p < 0.001$). We cannot tell a change of the provider’s model from a change of its default settings or of how the gateway routes the same identifier; to the user they are one event. The certifier measures the sum of these effects against the cache it is asked to reuse, which is the quantity a user is exposed to, and its certificates moved accordingly. The loose budget

Table 7: Further models on the lab column’s two edit pairs under the pinned procedure (presented-cells estimand, $\delta=0.1$; the same seeded rows as the primary evaluation vector, one fresh draw per cell, $T=0$). Each entry gives mean savings with the certification rate in parentheses, both in percent over 1,000 sampling replications of that model’s stored draws. The primary row repeats Table 2, with the column-level floor of the noise-floor paragraph. Here n is the rows requested; a few requests per model failed, leaving at least 1,966 usable rows per measurement. A rate that falls half way between two one-decimal values is printed to two decimals. Gemini 3 Flash ran these two pairs only: in its reasoning regime the remaining pairs would have been the costliest runs of the study, and they were left out.

Model	n	floor	formatting-only (v1→v2)			synonym rewording (v2→v3)		
			flip	$\alpha=0.1$	$\alpha=0.2$	flip	$\alpha=0.1$	$\alpha=0.2$
DeepSeek V4 Flash (primary)	2,000	5.0%	6.35%	82.6 (98.9)	88.1 (100.0)	8.90%	55.6 (81.4)	87.4 (99.9)
Gemini 2.5 Flash-Lite	2,000	4.85%	25.45%	0.0 (0.0)	0.1 (0.1)	31.5%	62.1 (100.0)	63.0 (100.0)
Gemini 3 Flash	2,000	0.4%	4.5%	82.1 (100.0)	83.2 (100.0)	6.2%	80.7 (99.2)	85.3 (100.0)
GLM-4.7-Flash	2,000	10.35%	25.0%	0.0 (0.0)	3.3 (70.7)	31.3%	0.7 (1.3)	54.5 (98.7)
Qwen3.7-Flash	2,000	5.45%	14.35%	0.5 (1.4)	51.5 (99.3)	27.2%	40.0 (94.5)	48.8 (100.0)
GPT-5 nano	2,000	34.8%	39.6%	0.0 (0.0)	0.0 (0.1)	39.7%	0.0 (0.0)	21.9 (65.4)
Gemini 3.8 Flash	2,000	0.95%	0.85%	92.6 (100.0)	95.5 (100.0)	4.9%	90.3 (100.0)	90.5 (100.0)

Table 8: The remaining three edit pairs on further models (same protocol and same entries as Table 7). Floors are each model’s self-flip rate on that column, measured in the same run; the primary model’s select-column floor is the $n=200$ probe of the noise-floor paragraph. The primary row repeats Table 2. Its widening-column floor is not shown, because the primary model’s self-flip rate under the production column’s first version was not measured (the lab column’s 5.0% is under the widened definition). A stratum smaller than the first look is drawn in full and cannot be certified: Gemini 2.5 Flash-Lite holds 5 cached-TRUE cells on the widening column.

Model	SO scope widening (production column)				Djinni seniority tier (four-way select)						
	floor	flip	$\alpha=0.1$	$\alpha=0.2$	floor	formatting-only (v1→v2)			criteria change (v2→v3)		
						flip	$\alpha=0.1$	$\alpha=0.2$	flip	$\alpha=0.1$	$\alpha=0.2$
DeepSeek V4 Flash (primary)	–	9.45%	66.3 (88.8)	86.6 (99.9)	6.5%	12.30%	8.4 (73.8)	52.1 (100.0)	12.45%	14.4 (82.8)	48.6 (100.0)
Qwen3.7-Flash	1.5%	17.15%	0.1 (1.5)	24.3 (99.3)	3.1%	5.85%	54.7 (99.9)	83.5 (100.0)	6.6%	39.9 (98.2)	83.2 (100.0)
Gemini 2.5 Flash-Lite	0.0%	8.6%	21.1 (45.8)	94.2 (99.5)	0.2%	6.5%	30.5 (99.9)	85.3 (100.0)	5.1%	53.5 (100.0)	86.1 (100.0)
Gemini 3.8 Flash	0.9%	4.6%	82.4 (95.9)	94.3 (100.0)	3.1%	4.15%	57.6 (100.0)	86.4 (100.0)	11.45%	24.9 (91.6)	56.4 (100.0)
GPT-5 nano	5.75%	36.55%	0.0 (0.0)	0.0 (0.0)	16.25%	15.8%	0.0 (0.1)	29.3 (99.8)	21.1%	0.0 (0.0)	18.7 (98.2)
GLM-4.7-Flash	9.4%	17.95%	46.6 (81.3)	72.6 (100.0)	4.4%	8.5%	20.1 (82.7)	80.4 (100.0)	9.3%	38.7 (99.9)	66.9 (100.0)

Table 9: Deployment-scale certification of the formatting edit on the 89,184-row column under the exact bound: one seeded sample path, two model snapshots. “FALSE look / flips” is the look at which the cached-FALSE stratum was decided and the flips observed there. The cached-TRUE stratum is refused at its first look throughout. Audited error is the flip rate among reused rows that hold an oracle cell of that snapshot.

Budget	oracle calls	FALSE look / flips	reused	savings	audited error (flips/rows)
<i>August 2026 snapshot: the stored oracle draws of the live run, replayed</i>					
$\alpha=0.2$	90	45 / 1	81,424	91.3%	3.15% (89/2,821)
$\alpha=0.1$	225	180 / 5	81,289	91.1%	3.16% (85/2,686)
$\alpha=0.05$	not reachable from the stored draws (699 further draws needed)				
<i>September 2026 snapshot: oracle drawn live, same cache</i>					
$\alpha=0.2$	90	45 / 2	81,424	91.3%	7.46% (104/1,395)
$\alpha=0.1$	405	360 / 20	81,109	90.9%	7.96% (86/1,080)
$\alpha=0.05$	135	90 / 6 (refused)	0	0.0%	–

certified again at look 45, for 90 calls, in 101 seconds of wall clock including the oracle calls. The tight budget needed 405 calls, one look deeper than on the August draws. The budget $\alpha=0.05$ was refused by the futility rule after 135 calls. The refusal is correct: the stratum’s flip rate against the aged cache exceeds that budget, so its cells are recomputed rather than served. Audited against every September oracle cell, the reused rows flip at 7.46% under the loose certificate and 7.96% under the tight one, inside both budgets and

well above the August figure. Appendix C gives the like-for-like drift comparison, the per-look evidence, and the run’s accounting.

Sample cost against table size. The schedule stops on evidence about a stratum’s rate, and its deepest look is fixed. The sample is therefore bounded whatever the table’s size, while the population a certificate covers grows with the table. At $\alpha=0.2$ the certification spent 90 calls on 89,184 rows, where the benchmark’s replications

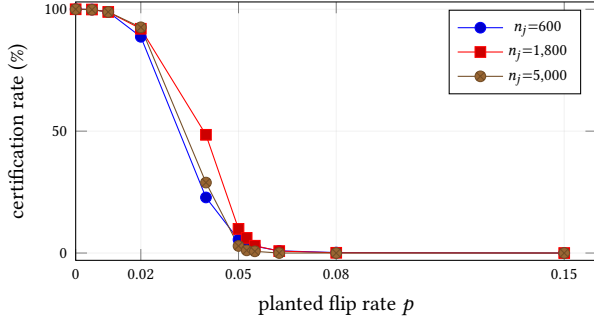


Figure 3: Calibration power at $\alpha=0.05$ (presented-cells estimand, 2,000 trials per point). Certification rises smoothly as p falls below α and is small but nonzero just above it. There the realized population rate of a planted stratum often falls below α , and certifying it is correct. The unconditional rate of certificates whose realized error exceeds α stays at or below 1.00% in every configuration, against the nominal 5%. The $n_j=1,800$ curve sits above the other two at intermediate p , because the exact bound’s finite-population correction tightens as the sampled fraction grows. The six-look schedule reaches most of an $n_j=1,800$ stratum but a minority of an $n_j=5,000$ one. The $n_j=600$ stratum’s final look is the whole stratum, which cannot certify, so its deepest certifying look is its fourth.

spend 90–180 calls (mean 97) on 2,000 rows. Savings therefore rose from 88.5% on the benchmark’s main seed to 91.3% at the scale of the corpus. Savings here are shares of rows; rows with identical bound content share one stored cell (Appendix C).

5.6 Validity checks

Calibration. The calibration study plants strata at known flip rates and runs the certifier on them (Figure 3). It covers 198 configurations: planted rates $p \in [0, 0.25]$, budgets $\alpha \in \{0.05, 0.1, 0.2\}$, three stratum sizes, and both modes, 396,000 runs in all. The quantity Theorem 3.1 bounds is the rate of unsafe certificates, certifications of a stratum whose realized flip count exceeds the budget. Its worst value over all configurations is 1.00%, against the per-stratum $\delta_j=5\%$ the study plants. Planting by rate is not the least favorable case, because the realized count of a planted stratum often falls on the safe side of the budget. The least favorable case is a fixed population whose flip count sits one flip past the budget, and there the probability of an unsafe certificate can be computed exactly. Over the 9 (budget, size) combinations of the grid it is at most 2.7% in the default mode and 1.1% in the strict mode, inside δ_j in each. The gap to δ_j comes from the Bonferroni split across looks, from the discreteness of the test, and from the futility rule, which stops sample paths that a later look might have certified. Appendix F gives the computation, the argument that this population is the least favorable one, and the value for each combination.

Independence across rows. Assumption 1 asks that oracle draws not interfere across rows, and provider-side batching could in principle correlate concurrent requests [21, 57]. We tested it on 300

rows of the evaluation vector. The formatting edit’s new version was drawn twice for every row against the same cache, once with a single request in flight and once at a concurrency of 32. A further concurrent arm of 1,000 rows was drawn a day later. Appendix D gives every test; three findings matter here.

First, among completed requests the flip rates agree: 8.9% sequential against 8.8% concurrent. That comparison could detect a difference of 7.0 percentage points, not a smaller one. The lag-one tests of serial dependence do not reject in any arm.

Second, failed requests track the serving conditions more than the concurrency. The first concurrent arm had three times as many failed or timed-out requests as the sequential arm, 28 against 9 of 300. The larger concurrent arm, a day later under faster serving, had none. The certifier counts a failed call as a flip (§4). Counted that way the three arms read 11.7%, 17.3%, and 7.1%. The first two share their rows, and their difference gives an exact McNemar $p=0.036$. The rule is conservative, so failures cost certificates, not validity.

Third, dependence among draws issued together is not excluded, and the planned test for it is confounded. Over blocks of 32 completions, the first concurrent arm’s flip counts have 2.08 times the variance that independent draws at one rate would give. The sequential arm, with one request in flight, shows excess on the same rows (a ratio of 1.39). And 11 rows flip in both of those arms where chance would give about 2. In the two paired arms the excess therefore follows the rows they share at least as much as the concurrency. The larger arm, on other rows, reads 1.26. Pooled over both concurrent arms the ratio is 1.42 (95% interval [0.94, 2.37]), and the one-sided test rejects narrowly. A simulation prices that ratio as if all of it were dependence, for the least favorable case, a stratum whose flip rate sits one flip past the budget. With independent draws the certifier errs there with probability at most 2.3%. At the pooled estimate it errs with probability 5.4%, and at the upper end of the pooled interval with probability 13.1%, against a nominal δ_j of 0.05. The first concurrent arm alone, eight blocks drawn under slow serving, is compatible with a much larger figure.

The check therefore supports less than independence. It does not show that concurrent draws interfere. It also cannot rule out, at the upper end of the pooled interval, an error probability two to three times the nominal one on a stratum that sits just past its budget. Three things limit that exposure. It is confined to such strata, and at $\alpha=0.2$ the deployment stratum’s audited error is far below its budget. The replication results of this article resample frozen labels, a fixed population for which the hypergeometric law holds however the labels were drawn, so only a live certification relies on the assumption. And a certification sample is small, so its calls can be issued one at a time at a cost in wall clock and none in calls; the sequential arm is too short to show that no dependence remains then. The check covers one model through one gateway at one concurrency, in the September snapshot. The September certificates of §5.5 were themselves drawn concurrently.

5.7 Ablations

The bound ablation. Every result above spends its δ through the exact bound of Eq. (1), split across looks by Bonferroni. The

bound, more than the stratification, sets the frontier. We measured how much by swapping it inside the same pinned procedure over the same stored labels (Table 10; Appendix B gives every configuration). Five bounds are compared: the exact bound; its binomial limit, the Clopper–Pearson bound [13]; the Maurer–Pontil empirical-Bernstein bound [37]; the Bardenet–Maillard without-replacement bound [6, 47]; and a Waudby-Smith–Ramdas betting confidence sequence [56], which needs no split across looks.

Across the 20 configurations of Table 2 (main seed), the exact bound certifies 10 of them, against 7 under Maurer–Pontil and 7 under Bardenet–Maillard, at higher mean savings and on fewer oracle calls. The empirical-Bernstein bounds pay a range term that a binary loss does not need. The betting sequence beats the exact bound on 1 of the 20 configurations, where the split across looks bites hardest, and loses to it on 2. Its validity for draws without replacement rests on a caveat that Appendix B states. All five arms were also replayed through the calibration protocol. In that replay the exact bound’s rate of unsafe certificates at planted nulls is at most 1.00% against a per-stratum δ_j of 5%. The exact bound is the pinned one because it is the bound the sampling model itself implies: it uses the hypergeometric law with no relaxation, where the empirical-Bernstein bounds relax it to cover losses a binary flip cannot take.

The stratification hierarchy. One stratum is too few: aggregate-only certification conceals the cached-TRUE subgroup’s error (baseline B2). More strata than the cached values are usually too many. Refining each value stratum by embedding-interaction tertiles keeps validity (Assumption 2) and costs power through the δ/K split, unless within-value risk is heterogeneous. On the headline configuration the refinement certifies less than value strata alone (82.7% against 88.1% mean savings on SO formatting at $\alpha=0.2$). Across the 40 shared configurations of the released ablation it is better in 6 and worse in 22. Every win sits at a budget where value strata alone certify in at most 2.7% of replications. We report this as a negative result. On these columns within-value risk is homogeneous enough that the split is repaid only where the value strata certify almost nothing.

Minority-stratum anatomy. The cached-TRUE stratum of the Boolean column flips 37.1% under a formatting edit. One candidate cause is cache noise, which a majority vote of three draws at write time would remove [55]. The stored draws do not support it. Voting on both sides (vote-of-3) brought the stratum’s flip rate to 24.7% in a dedicated probe and to 34.4% on the three-draw labels, two separate runs over the same 186 rows. The vote overturned 20.4% and 16.7% of the cached TRUE values, respectively. A cache built by the vote fares no better. Its TRUE stratum, the 188 rows whose three-draw majority is TRUE, has a lower floor (17.6% against 23.0%) but flips at 41.0% against fresh draws of the edited prompt (Table 4). Voting at write time cleans the cache without making the minority stratum certifiable under this edit. Refusing and recomputing that stratum is the response the data support.

6 The prompt-edit benchmark

Semantic benchmarks exist, but they benchmark *engines* on a fixed task [30]; none exercises maintenance across prompt versions, which needs a different artifact: version pairs plus per-cell flip labels against the new version. Ours comprises five components. The first is two redistributable corpora: the Stack Overflow Developer Survey (89,184 rows, ODbL 1.0 with the Database Contents License, DbCL 1.0) and Djinni English candidate profiles (210,250 rows, MIT). The second is three materialized columns of two types: two Boolean columns over structured facets, one of them the production column and one a lab replica seeded from its second version; and one four-way ordinal select over long free text. The third is five realistic edit pairs spanning formatting-only edits, synonym rewording, scope widening, and a criteria change. The fourth is per-cell flip labels for $n=2,000$ rows per version pair at pinned decode, the three-draw relabeling of the headline pair (which also supplies its self-flip floor), and the select column’s self-flip probe. The labels and the relabeling draws are 28,000 labeled cells in all: 16,000 distinct version cells and 12,000 relabeling draws. The fifth is the deterministic harness (seeded sampling, adaptive certifier, baseline policies) that reproduces every number in this article. Table 11 lists the five pairs with their exact deltas. Ledgered model spend across the entire evaluation is \$12.01 (plus 146,753 cells from direct-call probes the ledger does not see), so the benchmark is cheap to extend to budget-class models.

7 Related work

Table 12 positions sIVM against the lines of work it borrows from, is compared with, or is mistaken for, by the quantity each certifies and the axis of change it holds fixed.

Guaranteed semantic query processing. Semantic-operator systems bound the error of *one-shot* execution against a reference model. They include LOTUS’s optimized operators with accuracy targets [43], SUPG’s proxy-based selection with precision/recall guarantees [26], and its aggregation counterpart ABae, which stratifies by proxy before spending an expensive predicate [27]. They also include BARGAIN’s adaptive cascade thresholds [58], ThalamusDB’s approximate natural-language predicates with error bounds [25], and pipeline-level accuracy budgeting in Task Cascades [50]. All certify a *fixed* definition per query; none transfers evidence across a definition change. We reuse their statistical framing (a labeled sample, a bound, a threshold) in a maintenance loop those systems do not model, with a different bound. The loss is binary and the sample is drawn without replacement from a frozen stratum. The exact finite-population bound, the hypergeometric form of Clopper and Pearson’s binomial limits [13], is therefore available, and it dominates the empirical-Bernstein bounds that a general bounded loss calls for (\$5.7). Time-uniform confidence sequences [24, 56] are the alternative that would remove the split across looks. The certificate itself is distribution-free risk control in the sense of risk-controlling prediction sets [7] and Learn then Test [2]. It is an upper confidence bound on a risk that must clear a user-set α with probability $1 - \delta$, here evaluated per stratum and per look with the multiple-testing accounting those frameworks make explicit. The declarative LLM data systems that optimize cost and quality without a statistical target

Table 10: Five upper confidence bounds inside the same pinned procedure on the same stored labels. The first two columns give each bound on a clean sample of 90 draws and on the deployment evidence of 5 flips in 180 draws. The four fixed-sample bounds are read at the pinned per-look level, $\delta/12$, and the betting sequence at the unsplit per-stratum level, $\delta/2$ (Appendix B states the caveat on its validity). The next three columns give the certifying configurations, mean savings, and oracle calls over the Table 2 grid (main seed). The last gives the worst rate of unsafe certificates, the event of Theorem 3.1, at planted nulls in the calibration replay (per-stratum $\delta=0.05$). WoR: without replacement.

Bound	upper bound on		Table 2 grid (20 configurations)			unsafe at nulls worst
	0/90, $N=1,800$	5/180, $N=81,469$	certifying	mean savings	oracle calls	
Exact finite-population (pinned)	0.050	0.073	10	29.5%	7,458	1.00%
Clopper–Pearson (binomial)	0.052	0.073	9	28.1%	8,054	0.50%
Betting confidence sequence	0.065	–	9	29.1%	7,616	1.20%
Maurer–Pontil	0.144	0.140	7	18.5%	11,515	0.00%
Bardenet–Maillard (WoR)	0.317	0.230	7	16.4%	12,325	0.00%

The betting sequence has no 5/180 entry because its bound is a function of the whole draw sequence, not of (k, n) alone.

Table 11: The five versioned prompt-edit pairs. Deltas are transcribed from the stored prompt templates; bound fields are the row attributes the template interpolates. Flip rates are single-fresh-draw labels against the cached version on the $n=2,000$ evaluation vector (Table 2); the SO synonym pair is measured against the formatting version’s cells ($v2 \rightarrow v3$), as the benchmark defines it.

Pair	Column (corpus; contract; bound fields)	Exact delta	flip
SO formatting-only ($v1 \rightarrow v2$)	Data-platform specialist, lab replica (SO; Boolean; role, databases, platforms, tools, other tech)	Two paragraph breaks inserted after the question and before the “Judge from” sentence; “OR heavy analytics engineering” lowercased to “or heavy analytics engineering”; no content word changed	6.35%
SO synonym rewording ($v2 \rightarrow v3$)	same	“this person” $\rightarrow$ “this individual”; “works primarily on” $\rightarrow$ “chiefly builds and maintains”; “heavy analytics engineering” $\rightarrow$ “intensive analytics engineering”; “Judge from role ...and other tech” $\rightarrow$ “Decide using their role ...and other technologies”; “Working with multiple ...counts as evidence” $\rightarrow$ “Experience with several ...is supporting evidence”	8.90%
SO scope widening ($v1 \rightarrow v2$)	Data-platform specialist, production column (SO; Boolean; same fields)	Definition extended from “warehouses, lakes, pipelines, distributed data stores, or ML data tooling” by “OR heavy analytics engineering”; the exclusion “Merely using one relational database for app work does not qualify” replaced by the inclusion “Working with multiple analytical/distributed databases (BigQuery, Snowflake, Spark, Kafka, Elasticsearch) counts as evidence”	9.45%
Djinni formatting-only ($v1 \rightarrow v2$)	Seniority tier, lab (Djinni; four-way ordinal select; position, years of experience, CV text)	The one-line template broken into a question line, three labeled input lines (Position, Years of experience, CV), and the instruction sentence, with no word changed	12.30%
Djinni criteria change ($v2 \rightarrow v3$)	same	Sentence appended: “Evidence of team or people leadership (leading engineers, owning hiring, running a team) moves the candidate up one tier”	12.45%

are Palimpsest’s AI-powered analytics [34], DocETL’s agentic pipeline rewriting [48], Evaporate’s structured views over data lakes [3], and UQE’s sampling-driven queries over unstructured collections [14]. They compute the same kind of prompt-defined per-record outputs. None certifies which of its outputs survive a redefinition.

Sampling inspection and sequential tests. Statistically, a certification is a lot-acceptance decision. A stratum is a finite lot, a flip is a defect, and the exact bound is the hypergeometric calculation of

acceptance sampling by attributes [16], with the tolerated fraction set by the user and the decision spread over looks. The look schedule is a group-sequential design. We split the failure probability evenly across the looks, where the spending functions of that literature [29, 44] would allocate it unevenly. Risk-limiting election audits [32] share the shape of the whole procedure: sample until the evidence bounds the risk of a wrong outcome, and otherwise fall back to the full count, as a refusal falls back to recomputation.

Table 12: What neighboring systems certify, and across what. sIVM is the only row whose certified quantity is the reuse of cached outputs across a change of the prompt definition.

System or line	Certified quantity	Held fixed	Changes	Guarantee form
SUPG, ABae, BARGAIN, LOTUS, ThalamusDB, Task Cascades [25–27, 43, 50, 58]	precision, recall, or accuracy of one query’s answer against an oracle	the query definition	nothing (one-shot execution)	finite-sample bound per query
vCache [45]	correctness of serving a cached answer to a <i>new</i> query	the answer semantics	the incoming query	learned per-entry threshold at a user error rate
FreshCache, Evergreen [31, 35]	probability that a cached entry is stale (world drift); cost of verifying aggregates	the query definition	the world the answer describes	fitted staleness model per tier; confidence sequences
Stale-view cleaning, online aggregation [22, 28]	error of an answer served from a stale view; a running aggregate	the view definition	the base data	sampling bound on the answer
Classical incremental view maintenance (IVM) [8]	exact equality of the maintained view with recomputation	the view definition	the base data	algebraic (no sampling)
View adaptation after redefinition [20]	exact equivalence of the rewritten view	deterministic operators	the view definition	algebraic (no sampling)
Step-level reuse [40]	per-step equivalence under a perturbed request	the model	the request	deterministic per-step check
Guarantee-free semantic caches [5] (B3)	nothing; a hit is a similarity above a threshold	nothing	the incoming query	none
Per-output verification: assertions [49], regression suites [15], majority vote [55]	correctness of one output against a checker or a majority	the prompt	the output under test	per-output check, no population bound
Conformal and distribution-free risk control for model outputs: risk-controlling prediction sets, Learn then Test [2, 7]	a risk of one fixed predictor’s outputs on new inputs	the predictor (model and prompt)	the input	finite-sample bound from a held-out calibration set
sIVM (this work)	false-reuse rate of cached stochastic cells kept under the new definition, per stratum	the model snapshot and the data	the prompt definition	exact finite-sample bound per stratum and look

Verified caching. vCache certifies that a *new query* may reuse a cached answer under one fixed definition, with learned per-entry thresholds and user error rates [45]. Evergreen spends confidence sequences and early stopping to cut the cost of *verifying* semantic aggregates, not to decide reuse [31]. FreshCache approves a cache hit only when a learned staleness model puts the probability that the entry is stale below a per-tier error budget [35]. FreshCache is the closest in shape, since a per-partition budget gating reuse is exactly our skeleton, and it differs on the two axes that decide the guarantee. Its risk is *change-of-world* (open-web evidence aging under fixed query semantics) where ours is *change-of-definition*. Its per-tier quantity is a fitted model’s point estimate where ours is a finite-sample upper confidence bound that either clears α or refuses. Our B1 baseline shows why these mechanisms do not port: across a definition change the query-to-entry similarity that such caches threshold carries no per-cell flip signal (AUC 0.454 and 0.499). sIVM certifies *change-of-definition* transfer, the axis those systems hold fixed; the two compose (a certificate

is scoped to a snapshot, and change-of-world machinery decides when the snapshot itself expires).

Incremental and adaptive view maintenance. Sampling a stale materialized view to bound the error of answers served from it [28] is the closest classical relative of what we do, and the difference is what goes stale. There the data drifts under a fixed view definition; here the definition itself changes over fixed data. That is why that work’s estimators condition on updates and ours on a prompt delta. The look schedule itself is online aggregation’s template [22], a running estimate whose interval tightens as sampling proceeds, repurposed to certify a stratum rather than to answer an aggregate. Classical incremental view maintenance (IVM) propagates base-data changes through a fixed view definition [8], and view adaptation after redefinition [20] rewrites the materialization when the definition changes. Both assume deterministic operators, where a rewrite either preserves a subexpression or does not. Stochastic LLM cells break the premise. Our measurements show that even a semantically null rewrite flips cells at about the column’s

noise floor (6.35% against 4.5%, both measured against the stored cache), so “preserved” is not a property a rewrite can have here, only a rate one can bound. Streaming semantic systems recompile operators mid-stream but recompute outputs instead of certifying reuse of old ones [11, 12]. Streaming cascades certify on the *data-arrival* axis, where new tuples meet a fixed operator, and leave the definition axis open [33]. Given that distinction, sIVM is, to our knowledge, the first maintenance procedure whose unit of transfer is a *statistically certified set of stochastic cells*.

The nearest counterexample in shape is step-level reuse with lightweight verification and selective patching across perturbed requests [40], which does reuse computation when a request changes. It differs on exactly the property this article is about: its verification is a deterministic per-step check, so it carries no error budget and makes no population statement about the set it reuses. In-database prompt management [36] and versioned, provenance-tracked prompt views [9] supply the substrate this work assumes, not a competing mechanism. We take the framing of prompts as versioned views from that line and claim only the certification machinery built on top of it, whose reuse is of *outputs*, not of a model’s internal state.

Prompt sensitivity and regression. Prompt-delta analysis for assertion synthesis [49] anticipates our edit taxonomy but targets pipeline quality checks, not reuse certification; interactive LLM regression tools frame migration pain without guarantees [15]. Our finding that surface edit class can fail to predict flip rate (a synonym paraphrase flipping more cells than a formatting edit; a formatting edit indistinguishable from a criteria change) shows why measurement, not taxonomy, must carry the decision. Formatting changes that preserve meaning are known to move task accuracy widely, and differently across models [46]. Our per-cell flip rates, and their variation across models, are that sensitivity seen at the level of individual outputs. A served model can also change under a fixed name [10], which is the drift our September re-certification measured against a cache.

8 Discussion and limitations

Which budgets are usable, and for what. Table 13 sets out what each budget tolerates and what it buys. Most of the savings sit at $\alpha=0.2$ and $\alpha=0.1$, and neither is a small tolerance. A budget is a share of a certified stratum. On the running example the certified stratum is the cached-FALSE one, so the cells at risk are rows shown as FALSE that a recompute would show as TRUE: hidden positives. At the scale of the corpus the loose budget tolerates 16,293 of them, about twice the 7,715 rows the column marks TRUE at all. The tight budget still tolerates more than that number. The audits found fewer, but not few. Carried to all the reused cells, the error rates audited under the loose certificates come to about 2,600 hidden positives on the August draws and about 6,100 in September. Each is a row on which a recompute would differ, not a row known to be wrong, and a certificate promises only its budget.

A budget that loose is defensible where a missed positive is cheap and the column is advisory: a lead list or an enrichment sheet that a person keeps working through, or an exploratory filter that is recomputed before anything is decided on it. The alternative on offer there is today’s status quo, which reuses everything at the

population flip rate with no bound at all. The same budget is not defensible where a FALSE closes a case (a screen that rejects on it), or where the column feeds an aggregate or a downstream filter. There a tighter budget and the strict mode are the right setting, and Table 13 shows what they buy. At $\alpha=0.05$ the stable Boolean column still certifies in most replications, but for less than half the savings, and the strict mode certifies almost never. The less stable select column certifies nothing below $\alpha=0.1$, and little at it. At the scale of the corpus nothing below $\alpha=0.1$ certified: the August draws stop short of the look that $\alpha=0.05$ needs, and the September run refused it. Tight budgets are affordable in calls, because the sample is bounded: 405 calls at $\alpha=0.1$ in September, against 89,184 cells. What they cannot buy is certification of a stratum whose flip rate is near or above them. A practitioner should therefore read the frontier of Figure 2 against the column’s floor, and choose the budget the downstream use tolerates, not the one that maximizes savings.

Statistical scope. Bonferroni across strata and looks is sound but conservative, and \$5.7 prices it. The exact bound leaves a gap between its measured unsafe-certificate rate and the nominal δ . A betting confidence sequence, which needs no split across looks, beats it where that split bites hardest and matches or loses to it elsewhere. The without-replacement betting sequence of [56] or a time-uniform boundary [24] would keep the exact bound’s validity under without-replacement sampling while removing the split. A group-sequential calibration of the six looks [29, 44] is a third route. The finite sum behind the calibration study’s worst-case figures gives the exact error probability of any per-look levels, so the levels could be chosen to spend nearly all of δ_j , up to the discreteness of the test. Levels tuned to one budget would give up the reading of several budgets from one sample path. The default mode is also conservative for its own estimand. The presented-cells rate leaves out the k_j flips the sample observed, so certifying when $u_{j,t} n_j - k_j \leq \alpha n_j$ would bound it as well. We keep the simpler rule, which bounds the whole-stratum count too. E-value selection (the e-value Benjamini–Hochberg procedure) [54] composes with any of these. None of them is needed for the guarantees stated here. The independence of oracle draws across rows is an assumption that our check probes but does not establish (§5.6); a live certification under concurrency inherits whatever dependence the serving path introduces. Our stratifiers are first instances. The lexical variant is weak on broad edits, and the embedding variant’s signal exists only for semantic edits. Learned interaction models, or a cheaper model’s answer used as a proxy, are therefore an open lever with guaranteed validity (Assumption 2).

Sequences of edits. Reuse is one edit deep in the released system. A cell reused under a certificate is recomputed at the next edit (§4), so savings do not accumulate over a series of edits: the edit after a certified one recomputes every cell that one reused. Theorem 3.1 does not force that choice. A later certification could test the carried-forward values against fresh draws of the newest version, because the guarantee concerns the stored values whatever version computed them. We have not evaluated sequences of edits, and each certification in a sequence spends its own δ .

Table 13: What each budget tolerates and what it buys. Tolerated cells are α times the 81,469 cells of the corpus-scale cached-FALSE stratum, also given as a multiple of the 7,715 rows the column marks TRUE. The corpus-scale columns are the live September certification of Table 9. The last six columns give the certification rate and the mean savings of the formatting-only edit, in percent over 1,000 replications: on the Boolean column in both modes, and on the select column in the default mode (Table 2; Appendix G).

Budget	tolerated, corpus scale		corpus-scale run (September)			Boolean, default		Boolean, strict		select, default	
	cells	$\times$ TRUE rows	calls	reused	audited	cert.	savings	cert.	savings	cert.	savings
$\alpha=0.2$	16,293	2.1	90	81,424	7.46%	100.0	88.1	100.0	88.1	100.0	52.1
$\alpha=0.1$	8,146	1.1	405	81,109	7.96%	98.9	82.6	98.7	80.6	73.8	8.4
$\alpha=0.05$	4,073	0.5	135	refused	–	74.4	38.6	4.5	3.4	0.0	0.0

Equivalence and columns. In the main evaluation equivalence is exact match, and the columns are two Boolean columns and one four-way ordinal select; numeric and free-text columns, which the introduction’s product examples also serve, are not evaluated there. The five author-written edit pairs, one per class per column, are enough to show that surface class can fail to predict flip rate, not to estimate how often it does. The edit-class findings should be read at that strength. Because both Boolean columns ask one question over one corpus, the observation that a column’s own stability dominates the edit rests on two columns.

The equivalence relation is the user’s to state. Theorem 3.1 needs only that it yield a binary flip indicator, so a graded column can take a tolerance in place of exact match, and §5.2 measures one on the seniority column. A numeric column would take a tolerance in the same way; none is evaluated here. A free-text column needs a judge, and the judge’s own error enters the budget. In a stratum, let q be the share of pairs the judge calls equivalent although a reader would not (missed flips). The flip rate the judge reports for the stratum is then at least the true rate minus q , and q is at most the judge’s miss rate among the pairs it calls equivalent. So let $\hat{m}$ be an upper confidence bound on that conditional miss rate at level δ_{cal} , from human-labeled pairs of the same column, and certify at $\alpha - \hat{m}$. A certified stratum’s true rate is at most α whenever the calibration bound and the certificate both hold, provided the judge’s conditional miss rate in the certified stratum is no higher than in the calibration pool. They both hold with probability at least $1 - \delta - \delta_{\text{cal}}$. Errors in the other direction (calling equivalent cells different) only raise the reported rate and cost power. The calibration belongs to the (edit, column) pair it was drawn from, or to a pool of comparisons on one column if their conditional miss rates are taken as equal, since a judge’s miss rate is a property of the outputs it compares. The machinery of this article is otherwise unchanged, but its power under a noisy judge has one measurement so far.

A free-text pilot. We ran the recipe once, on a free-text column over 2,000 SO profiles that names each respondent’s primary technical specialty in a short phrase, with a formatting-only edit and a synonym rewording. The judge is the primary model, asked in both orders whether two phrases name the same specialty. Its miss rate was calibrated on 201 pairs labeled by the author, and a second model labeled the same pairs as a check. The result is negative on power. Under the judge the column’s own floor, two draws of the same prompt, is 40.1%. That is above every budget, and the certifier

refused every stratum of both edits. The calibration itself is clean on the author’s labels: the judge missed 0 of the 150 pairs it called equivalent, which takes 2.0 percentage points off each budget. It is not clean on the second labeler’s labels, and one human labeler cannot settle which reading is right. The pilot therefore shows two things. The machinery and the accounting carry over to a judge unchanged. And a judge strict enough to calibrate cleanly can cost all the power. Appendix E reports the pilot in full.

Models and snapshots. One budget-class reasoning model carries the full experimental program. A further 6 models were run on the lab column’s two pairs and 5 of them on all five (§5.4). On 6 of the 6 at least one edit certified in nearly every replication at the loose budget, and the same edit’s flip rate ranged widely. The quantitative frontier (where savings sit for a given α , and which edits are benign at all) is therefore specific to the (edit, model) pair. The qualitative structure nevertheless held on every model: a floor that is measurable on the lab column, which limits what certifies when an edit leaves the output distribution unchanged, and a certifier whose refusals track the measured flip rate. Certificates are scoped to the model snapshot their oracle draws came from. §5.5 shows what that scope is worth. The same model identifier through the same gateway had drifted 55 days after the deployment run. The certifier measured the drift as a higher flip rate against the old cache. It re-certified the two budgets the August run had certified, one of them a look deeper than the exact bound’s replay of the August draws. It refused $\alpha=0.05$, on which the August evidence is silent. Detecting, without a full re-certification, when a snapshot has moved, i.e., composing sIVM with change-of-world invalidation [31, 35], is the natural next step.

Economics. The economic frame that matters in production is time-to-consistency under provider rate limits. In that frame certified maintenance samples 5%–74% of cells across the configurations of Table 2 that certify in most replications (5% on the headline formatting edit at $\alpha=0.2$) and recomputes only the strata it refuses, instead of recomputing 100%. That is a wall-clock and rate-limit gain that grows with table size, because sample sizes do not. The live re-certification of §5.5 measured the certification decision at 101 seconds of wall clock for the loose budget. This article does not measure two things. The certificate surface in the UI was not studied with users, so whether a person acts differently on a column that shows one is open. No claim here depends on the answer: the guarantee is a statement about the cells, whatever the interface

shows, and a user study is human-subjects work of its own. And the cost figures come from one ledgered materialization and one priced maintenance replay at the gateway’s list rates. Each further live measurement would cost a full recomputation of the corpus, so the figures show the scale of the saving, not a cost model.

9 Conclusion

Prompt-defined columns are materialized views whose *definition* changes far more often than their data, and the systems serving them today choose between recomputing everything and reusing everything. sIVM shows the middle is reachable and cheap: freeze a stratification, sample on a doubling schedule, bound each stratum’s flip rate with an exact finite-population bound, reuse only what clears the budget. At the scale of a full corpus and a budget of $\alpha=0.2$, that cost 90 oracle calls to certify reuse of 81,424 of 89,184 cells. The sample did not have to grow with the table, because the schedule stops on evidence about a *rate*, at a cost bounded independently of the size. These results come from one primary model on Boolean and select columns. On 6 of 6 further models, from four providers, at least one edit certified, and which edits did so varied by model.

The result we did not anticipate is that the certifier doubles as a measuring instrument for the column it maintains. A stratum refused at every budget is a slice of the table on which the model does not reliably reproduce its own answers, identified by a futility stop at the cost of one look. A flip rate that has risen on the same rows against the same cache 55 days later is a model that has moved under it. The certifier reports the movement in the currency the user pays, oracle calls per certified budget. Certified maintenance is therefore useful even where it certifies nothing: the refusal identifies, cheaply, reuse error that would otherwise go unmeasured.

Availability and reproducibility

What is released. The system, the certifier, every experiment script, and every result artifact are public in a single repository, <https://github.com/arjunlohan/project-lore>, under the PolyForm Noncommercial License 1.0.0, a source-available license that permits research and other noncommercial use. The release contains ingestion scripts for the two corpora, the five versioned edit pairs, the per-cell flip labels, the certifier, the adapter conformance suite, and every script and result artifact that produces a number in this article. The corpora are the 2023 Stack Overflow Developer Survey [52] and the English candidate profiles of the Djinni Recruitment Dataset [17]. Both are public and redistributable (under ODbL 1.0/DbCL 1.0 and MIT), and both are used as distributed by their maintainers under those licenses. The Djinni corpus is released anonymized at source, and this work collects no new human-subject data, attempts no re-identification, and releases only per-cell labels and aggregate statistics. Nothing in the evaluation path is withheld. Anyone rerunning the evaluation needs a model endpoint of their own; the labels are released so the statistical results can be reproduced without one.

Generated numbers. Every quantitative claim in this article’s prose, tables, and figures is emitted by a single generator (`scripts/experiments/gen-paper-assets.ts`) from the result artifacts and from the system’s cost ledger, and cited through

macros, never typed inline. Re-running an experiment therefore regenerates the sentences that cite it. Several numbers (the per-stratum floors, the ledger economics, the materialization wall clock, the certificate’s stamp count) are computed from the running system’s database, not from a result file. The generator dumps every one of those database reads to an artifact alongside the others, so the release is self-contained, with no dependence on our instance. A checker (`scripts/check-paper-numbers.ts`) enforces the macro discipline mechanically and fails on ten classes of defect, listed in Appendix A.

Hazards and the preprint. Three reproducibility hazards we hit are documented in the harness, and Appendix C describes them. An earlier version of this article is public as a preprint (Zenodo, doi:10.5281/zenodo.21833641); it is “the preprint” wherever this article says so. Relative to it, the certifier’s default bound changed from Maurer–Pontil to the exact finite-population bound and every number was regenerated. The preprint’s certification results are reproducible from the same release with the Maurer–Pontil arm selected.

A The mechanical guards

The checker (`scripts/check-paper-numbers.ts`) fails on any of ten conditions.

- (1) A hand-typed quantity in the prose outside an enumerated allow-list (user parameters such as α and δ , protocol constants such as the schedule looks, and generic thresholds quoted as such).
- (2) A quantity hardcoded inside the generator instead of derived from an artifact.
- (3) A macro defined but never cited, since an uncited macro can drift undetected.
- (4) A macro whose trailing space is swallowed into the next word, which is invisible in the source.
- (5) A sentence that asserts two quantities differ while citing two macros whose values are equal.
- (6) A spelled-out multiplier (“twice”, “an order of magnitude”) that the cited macros’ ratio does not support.
- (7) A containment claim (“inside”, “at most”) that the cited range and scalar macros do not support.
- (8) An absolute claim (“never”, “every”, “anywhere”) appearing in the abstract or the contributions list without an entry in an enumerated allow-list. This is the one class of defect the numeric guards are blind to by construction.
- (9) A scope claim (“on the same column”) whose cited macros come from different columns, checked against a provenance file the generator emits from the same pair definitions the experiments run on.
- (10) A sentence of the prose, of a list item, of a caption, or of the abstract longer than 60 words (table bodies, displayed mathematics, and the theorem environments excluded), a readability bound.

The generator adds guards of its own. It asserts the relations that §5, §8, and the appendices state between figures: in the paragraphs on the main results, the strict mode, the stratifier ablation, the matched competitor, the model study, the independence check, the usable budgets, the graded relation, and the pilot. It also asserts

that the separate audit verifier reproduces each audit the article prints, and that the released certify action is configured as the pinned procedure. It stops if a regenerated artifact breaks one. It rounds every figure half up on its decimal value, and rounds upward a figure quoted as a maximum (“at most”). Generating the quantities is necessary but not sufficient, because the sentences that *relate* quantities go stale independently of the quantities themselves.

B The bound ablation in detail

The bounds compared. Over the 20 configurations of Table 2 (main seed), the exact bound’s mean savings are 29.5% against 18.5% under Maurer–Pontil and 16.4% under Bardenet–Maillard, on 7,458 oracle calls against 11,515 and 12,325. Relative to the exact bound, Maurer–Pontil loses 3 certificates and gains 0, and of the shared ones it reaches 1 at the same look and 6 at a later one. The widest single gain is SO scope widening at $\alpha=0.1$, from 18.1% savings for 1,485 calls to 72.1% for 405. Of the two empirical-Bernstein bounds, the without-replacement form pays more, not less, because its constants are larger. The preprint reported it as tighter. Its replay used the Maurer–Pontil linear constant inside the Serfling form, and the published constant, $\kappa = 7/3 + 3/\sqrt{2}$ [6], is what the numbers here use. The Clopper–Pearson limit certifies 9 configurations at mean savings 28.1% on 8,054 calls, the finite-population correction buying the exact bound its one further certificate. The betting sequence certifies 9 configurations at mean savings 29.1% on 7,616 calls. It beats the exact bound on 1 of the 20 configurations, where the schedule’s Bonferroni split bites hardest (Djinni criteria change at $\alpha=0.2$: 76.3% against 62.8%). It loses to it on 2 and matches it on the rest (certificates lost 1, gained 0). Its validity here rests on a capital process built for draws whose conditional mean is constant, which without-replacement draws do not have. The without-replacement variant given in [56], which we did not implement, or a time-uniform boundary [24] would remove the split without that caveat and is the remaining power lever we see.

Validity comes before power, so all five arms were replayed through the calibration protocol, in a run separate from and smaller than the calibration study of §5.6 (36,000 trials per arm, 18,000 of them at planted rates above the budget). At planted nulls the exact bound certifies up to 6.3% of trials, against a per-stratum δ_j of 5%. Most of those certificates are correct, because the realized count of a planted stratum falls below αn_j in many trials. The certificates whose realized count exceeds the budget, the event of Theorem 3.1, are at most 1.00% of trials in any null configuration ($\alpha=0.1$, $p=0.105$, $n_j=1,800$; Wilson [0.54, 1.83]%). The certification rate at a planted null is not the theorem’s event, and it may exceed δ_j without a violation. In 0 of the 18 null configurations does its interval lie wholly above δ_j . The Clopper–Pearson limit certifies 1.5% of null trials at worst (0.50% unsafe), and the betting sequence 2.5% (1.20% unsafe). Maurer–Pontil and Bardenet–Maillard certify 0.0% and 0.0% (0.00% and 0.00% unsafe), leaving almost all of their δ_j unspent. A clean 1,800-cell stratum at $\alpha=0.05$ costs the exact bound 90 draws (180 at $n_j=5,000$ in the calibration study). That figure is a tie: at exactly that size the bound on a clean look of that many draws equals the budget, and a stratum a few cells larger

pays the next look. It costs the Clopper–Pearson limit 180, the sequence 180, Maurer–Pontil 360, and Bardenet–Maillard 720. The frontier in Figure 2 is therefore a property of the bound.

Configuration by configuration. Table 14 gives, for every configuration of Table 2 and each of the five bounds, the certified savings of the main seed and the oracle calls spent. Relative to the exact bound, the Clopper–Pearson bound certifies all but one of its configurations (lost 1, gained 0), reaching 7 of them at the same look, 0 earlier, and 2 later, since it ignores the finite-population correction. Maurer–Pontil reaches 1 at the same look, 0 earlier, and 6 later. Bardenet–Maillard loses 3 certificates, gains 0, and reaches 0 at the same look, 0 earlier, and 7 later. The betting sequence reaches 7 at the same look, 1 earlier, and 1 later.

C The deployment runs in detail

This appendix gives the evidence behind §5.5: the materialization, the certificate the system applied in August, the replay of its draws under the exact bound, the drift measurement, the September certification, and the call counts with three reproducibility caveats.

The materialization. The column’s 78,956 billed cells cost \$9.31 at \$0.000118 per cell. The run took 7.2 hours of wall clock at its 32-way concurrency, measured from its first cell write to its last. An estimate from stored latencies agrees: the column’s 80,902 computed cells, this run’s and those of earlier passes, average 10.5 seconds per call, and those latencies, summed and divided by the concurrency, give 7.4 hours. Rows with identical bound content share one stored cell, which is why the billed cells are fewer than the rows. The cells this article counts as sampled, reused, or recomputed are rows, and its savings are shares of rows.

The applied August certificate. Theorem 3.1 does not cover the live August run, which used the Maurer–Pontil bound; it is reported because it is the certificate the system applied (Table 15). Under Maurer–Pontil, at $\alpha=0.2$ the procedure spent 225 oracle calls to reuse 81,289 cells (91.1% fewer calls). It certified the cached-FALSE stratum at look 180 with 5 flips. Audited error was 3.16% over 2,686 audited reused rows. It refused the 7,715-row cached-TRUE stratum and so left for recomputation the 7,670 of its cells that the futility stop had not already drawn fresh. At $\alpha=0.1$ it escalated to 765 calls (25 flips in 720) for 90.5% savings. The run applied the loose certificate, persisting a reuse_certificate row and stamping its identifier on 79,130 of the 81,289 reused cells, which is what the table then served. The 2,159 it did not stamp already held a freshly computed cell when the certificate was applied, and the apply step refuses to write over one. Under the Bardenet–Maillard bound the loose certificate’s evidence (5 flips in 180 draws from the 81,469-row cached-FALSE stratum) reads 0.230, which does not clear the budget at the look at which it was issued. The exact bound reads 0.073 on the same evidence, which clears $\alpha=0.2$ and $\alpha=0.1$ alike. The tight certificate’s 25 flips in 720 read 0.055 under the exact bound, which clears $\alpha=0.1$, and 0.099 under Bardenet–Maillard, which clears it.

The August draws under the exact bound. Replayed over the stored August draws at the loose budget, the exact bound certifies the cached-FALSE stratum at look 45 (flips 1, bound 0.143). The

Table 14: Certified savings of the main seed (oracle calls in parentheses) for every configuration and bound; “refused” gives the calls spent before the stop.

Edit	α	exact	Clopper-Pearson	betting	Maurer-Pontil	Bardenet-Maillard
SO formatting-only	0.02	refused (225)	refused (225)	refused (225)	refused (225)	refused (225)
SO formatting-only	0.05	18.7% (1,485)	18.7% (1,485)	18.7% (1,485)	refused (1,485)	refused (1,485)
SO formatting-only	0.10	88.5% (90)	86.2% (135)	86.2% (135)	54.7% (765)	54.7% (765)
SO formatting-only	0.20	88.5% (90)	88.5% (90)	88.5% (90)	86.2% (135)	72.7% (405)
SO synonym rewording	0.02	refused (90)	refused (90)	refused (90)	refused (90)	refused (90)
SO synonym rewording	0.05	refused (90)	refused (90)	refused (90)	refused (90)	refused (90)
SO synonym rewording	0.10	refused (90)	refused (90)	refused (90)	refused (90)	refused (90)
SO synonym rewording	0.20	82.3% (225)	82.3% (225)	82.3% (225)	82.3% (225)	73.3% (405)
SO scope widening	0.02	refused (90)	refused (90)	refused (90)	refused (90)	refused (90)
SO scope widening	0.05	refused (765)	refused (765)	refused (765)	refused (765)	refused (765)
SO scope widening	0.10	72.1% (405)	72.1% (405)	72.1% (405)	18.1% (1,485)	18.1% (1,485)
SO scope widening	0.20	87.8% (90)	87.8% (90)	87.8% (90)	81.1% (225)	72.1% (405)
Djinni formatting-only	0.02	refused (180)	refused (180)	refused (180)	refused (180)	refused (180)
Djinni formatting-only	0.05	refused (315)	refused (315)	refused (315)	refused (315)	refused (315)
Djinni formatting-only	0.10	19.2% (495)	refused (878)	refused (878)	refused (878)	refused (878)
Djinni formatting-only	0.20	52.4% (953)	52.4% (953)	52.4% (953)	19.2% (1,617)	19.2% (1,617)
Djinni criteria change	0.02	refused (180)	refused (180)	refused (180)	refused (180)	refused (180)
Djinni criteria change	0.05	refused (315)	refused (315)	refused (315)	refused (315)	refused (315)
Djinni criteria change	0.10	18.9% (540)	18.9% (540)	18.9% (540)	refused (918)	refused (918)
Djinni criteria change	0.20	62.8% (745)	54.4% (913)	76.3% (475)	27.9% (1,442)	18.9% (1,622)

Table 15: The Maurer-Pontil certifications on the deployment column, kept for the record. Theorem 3.1 does not cover them (§3). The first row is the certificate the system applied in August 2026. Columns as in Table 9.

Budget	oracle calls	FALSE look / flips	reused	savings	audited error (flips/rows)
<i>August 2026 snapshot: the live run</i>					
$\alpha=0.2$	225	180 / 5	81,289	91.1%	3.16% (85/2,686) (applied)
$\alpha=0.1$	765	720 / 25	80,749	90.5%	3.03% (65/2,146)
<i>September 2026 snapshot: the draws of Table 9</i>					
$\alpha=0.2$	225	180 / 11	81,289	91.1%	7.54% (95/1,260)
$\alpha=0.1$	1,485	1,440 / 106 (refused)	0	0.0%	–

maintenance workload is then 7,760 cells against 89,184 for a full recompute. At $\alpha=0.1$ it certifies at look 180 (5 flips, bound 0.073) for 225 calls, the count the Maurer-Pontil run needed for the loose budget, reusing 81,289 cells (91.1% savings). At $\alpha=0.05$ the schedule’s deepest look reaches 1,440 rows of the stratum, 699 of which hold no August draw. Realized error was audited against oracle-computed cells of that snapshot only (status done/cached). Certificate copies are excluded by construction, as verbatim v1 values would register as non-flips. Of the 2,821 audited reused rows, 1,812 belong to the released $n=2,000$ evaluation vector and show 3.20% on their own. The released subset is a consistency check with Table 2, and the remainder is independent evidence. Priced at the ledger’s measured \$0.000118 per cell, the maintenance workload would cost \$0.91 and about 42 minutes of wall clock on the latency estimate, against the \$9.31 the corpus cost to materialize. That estimate uses the materialization’s mean per-call latency at the same 32-way concurrency. We price the recompute leg without executing it.

The drift measurement. The September oracle draws are stored under a version of their own, so draws of the two snapshots never mix. Reasoning effort is left at the provider’s default, one of the settings whose change we cannot tell from a change of the model. Of the 720 cached-FALSE rows the August run had sampled, 36 flipped only in September against 11 only in August, the counts

behind the McNemar test of \$5.5. A like-for-like comparison separates the model from the edit. The v1 prompt re-drawn in September disagreed with its own August draws, the cache, on 4.4% of those rows (Wilson [3.2, 6.2]). The same prompt’s within-snapshot self-disagreement is 2.6% ([2.0, 3.1]). The v2 prompt re-drawn disagreed with its August draw on 6.5% ([4.9, 8.6]). The edit’s flip rate with both sides drawn in September was 5.8% ([4.3, 7.8]). The ledger shows the same change: 534 output tokens per cell against 290 in August, at 24 seconds per call against 9. On the 45 cached-TRUE rows the edit’s flip rate read 28.9% in August and 24.4% in September. Of those rows, 8 flipped only in August and 6 only in September (McNemar $p=0.79$). The v2 prompt re-drawn disagreed with its August draw on 31.1% of them ([19.5, 45.7]). The same stratum’s floor on the lab column is 23.0%. These rows are too few to resolve a shift.

The September certification. At $\alpha=0.2$ the certifier certified the cached-FALSE stratum at look 45 (2 flips, against 1 in August) and reused 81,424 cells (91.3%). The cached-TRUE stratum was refused at look 45 with 11 flips. At $\alpha=0.1$ it escalated to look 360 (20 flips, bound 0.091) and reused 81,109 cells (90.9%). The 315 calls beyond the loose arm’s 90 took a further 304 seconds. At $\alpha=0.05$ the futility rule refused the stratum at look 90 (6 flips, 6.7%). Under Maurer-Pontil the same September draws certify the loose budget after 225 calls and are refused at the tight budget after 1,485 calls (Table 15). The audits cover 1,395 rows under the loose certificate and 1,080

rows under the tight one. The sample path needed 1,485 September oracle cells across the arms. Of these, 699 were already held from a pass earlier that day and 786 were requested from the runner. Of the requested cells, 20 were served by the content-hash cache and 766 were drawn fresh for \$0.15. That is \$0.000199 per cell against \$0.000118 in August. The longer outputs are the difference. The drift measurement drew 752 more.

Call counts and reproducibility. The call count of a certification is itself random, since a schedule stops at the first look whose bound clears α . At $\alpha=0.2$ the benchmark’s main seed spent 90 calls, with a replication median of 90, and at $\alpha=0.1$ the replications span 90–405 calls (mean 188, median 225). Three reproducibility caveats apply. In MySQL the projection list changes which rows a seeded ORDER BY RAND returns, so the evaluation sample is defined by an exact query string, not by the seed alone. A linear-congruential generator written with JavaScript number multiplication silently loses low bits, so all sampling uses exact 32-bit arithmetic. And the certifier at the time of the August run shuffled each stratum in the order the cell query returned, which the query planner served through one index in August and another afterwards. The August run walked the content-hash order, the certifier now sorts by content hash before shuffling, and the replay reproduces every count of the August run under the Maurer–Pontil arm (the released script asserts it).

D The independence check in detail

Table 16 collects the statistics of the check that §5.6 summarizes.

The check. Each of the 300 rows had the formatting edit’s new version drawn once with a single request in flight and once at a concurrency of 32 (§5.6). The sequential arm took 78 minutes of wall clock and the concurrent arm 4.6. The planned comparison excludes requests that errored or exceeded a 90-second ceiling, and the ceiling binds more often under concurrency. The released certifier does not exclude them: it counts a failed call as a flip. Under that rule the sequential arm reads 35 flips of 300 and the concurrent arm 52 (two-proportion $p=0.049$). Of the rows, 21 count as flips only sequentially and 38 only concurrently (exact McNemar $p=0.036$). A certifier under concurrent load therefore sees more flips than one that issues its calls one at a time, which makes it refuse more often and never certify more. The larger arm, drawn under faster serving, had no failed request. On completed requests the two arms’ rates give a two-proportion $p=0.96$. On the 263 rows both arms kept, the rates are 9.5% sequential and 8.4% concurrent. Of those rows, 14 flipped only sequentially and 11 only concurrently (exact McNemar $p=0.69$). The excluded rows show no sign of being flip-prone, on small counts. Of the 28 rows the concurrent arm dropped, 1 flipped in the sequential arm. Of the 9 the sequential arm dropped, 2 flipped in the concurrent arm.

The planned tests. The script fixed three analyses before the run, each reported against its reference distribution at the 5% level. The first is a two-proportion test on the flip rates, whose minimum detectable difference at 80% power is 7.0 percentage points. The second sets the two arms’ disagreement on the same row against the August stratum floors. The third looks for serial dependence

in completion order, with a lag-one permutation test and a dispersion test of flip counts over consecutive blocks of 32 completions. The two arms disagree with each other on 9.5% of rows ([6.5, 13.7]): 9.2% of 238 cached-FALSE rows and 12.0% of 25 cached-TRUE rows, against August floors of 2.6% and 23.0%. After the drift measured in §5.5, that second comparison cannot separate concurrency from the change of snapshot, and we draw nothing from it. Table 16 gives the third analysis. In the concurrent arm the dispersion ratio’s 95% confidence interval (CI) is [0.91, 8.62], which covers 1, while its upper-tail chi-square test rejects.

Reading the tests. The rate and lag tests do not reject. The dispersion test rejects one-sided at the 5% level in the concurrent arm, on 8 blocks. To add blocks we drew a concurrent arm alone on a further 1,000 rows of the vector. Its block dispersion ratio has a 95% CI of [0.81, 2.26], and its flip rate set against the sequential arm’s gives $p=0.30$. This arm was drawn a day later, on other rows, and under faster serving (a median response of 3.7 seconds against 14.4 in the first concurrent arm), so the comparisons between arms also span a day of the snapshot. Pooled over both concurrent arms, the dispersion statistic is 52.5 on 37 degrees of freedom ($p=0.048$), a ratio of 1.42 with a 95% CI of [0.94, 2.37]. The block-scale question is therefore open. The larger arm alone does not reject. The pooled test, which includes the first arm, rejects narrowly. The sequential arm’s ratio does not reject either, on 9 blocks, which are too few to show that one-at-a-time draws are free of dependence. A block holds about three expected flips, where the chi-square reference is approximate. Permutation p-values for the same statistics, from shuffles of each arm’s own flip sequence, differ from the chi-square ones by at most 0.02 and fall on the same side of the 5% level in every case.

Three looks that were not in the plan argue for caution. Flips concentrate in slow responses: in the larger arm, 59 of the 500 slower responses flip against 12 of the 500 faster ones. Completion order is therefore not neutral with respect to the outcome. With blocks formed in launch order instead, the dispersion ratio is 1.61 in the larger arm ($p=0.019$) and 1.82 in the first ($p=0.08$). The ordering was chosen after seeing the data, so we report these as cautions, not as findings. The third look asks where the excess of the first concurrent arm comes from. Its flips concentrate late in its completion order: 4 in the first 136 completions and 20 in the last 136 (two-proportion $p < 0.001$). The sequential arm walks the same rows in the same order and leans the same way, 9 flips in its first 145 completions and 17 in its last 146. Of the 263 rows both arms kept, 11 flip in both, where independent rows at those rates would give about 2. The larger arm, on other rows, shows no such trend (34 and 37 flips in its two halves of 500). Block excess in the paired arms is therefore at least partly a property of their rows and the order those rows were issued in. A test in completion order cannot separate that from interference between requests.

The simulated cost of dependence. The simulation behind the error probabilities of §5.6 runs the certifier on the deployment stratum, at its two certified budgets and at a flip rate one flip past the budget, 100,000 times per setting. Draws arrive in blocks of 32 with beta-binomial dependence inside a block, so that block counts have a chosen variance ratio against the binomial. It is a model of dependence, not a measurement of it. Table 17 gives the error probability

Table 16: The independence check: one sequential arm and two concurrent arms of the formatting edit’s new version, each against the same cache. Lag-one autocorrelation and block dispersion are computed in completion order, the latter over blocks of 32 completions against its binomial value. The launch-order dispersion and the failure-as-flip rate were not in the plan. Flip-rate intervals are 95% confidence intervals.

	Sequential	Concurrent	Concurrent, larger arm
Run date	29 September 2026	29 September 2026	30 September 2026
Rows requested	300	300	1,000
Requests that failed (error or time ceiling)	9	28	0
Flip rate, completed requests (Wilson interval)	8.9% [6.2, 12.8]	8.8% [6.0, 12.8]	7.1% [5.7, 8.9]
Flip rate, failed requests counted as flips	11.7%	17.3%	7.1%
Lag-one autocorrelation (permutation p)	0.070 (0.27)	−0.006 (0.95)	−0.031 (0.34)
Block dispersion ratio (blocks; upper-tail p)	1.39 (9; 0.20)	2.08 (8; 0.04)	1.26 (31; 0.15)
Launch-order dispersion ratio (p)	–	1.82 (0.08)	1.61 (0.019)

Table 17: Simulated probability that the certifier errs on a stratum one flip past its budget, by the dispersion ratio of flip counts within blocks of 32 draws (deployment stratum; nominal per-stratum level 5%).

Source of the dispersion ratio	ratio	$\alpha=0.2$	$\alpha=0.1$
Independent draws	1.00	2.3%	1.9%
Larger concurrent arm, estimate	1.26	4.1%	3.5%
Both concurrent arms pooled, estimate	1.42	5.4%	4.7%
Larger arm, blocks in launch order (unplanned)	1.61	7.1%	6.1%
First concurrent arm, estimate	2.08	10.8%	9.6%
Larger arm, upper end of its interval	2.26	12.2%	10.8%
Both arms pooled, upper end of the interval	2.37	13.1%	11.7%
First arm, upper end of its interval	8.62	37.9%	38.8%

at every ratio the check reports. With independent draws it is at most 2.3%. At the pooled estimate it is about the nominal level, and at the upper end of the pooled interval it is between two and three times nominal. The first concurrent arm’s own interval, from 8 blocks, reaches ratios at which the certifier would err far more often. At the loose budget the deployment stratum is not one whose flip rate sits just above the budget: its audited error is 3.15% in August and 7.46% in September (Table 9). At the tight budget the September audit, 7.96% against $\alpha=0.1$, leaves less room.

E The free-text pilot

Setup. The pilot column of §8 has three prompt versions: a baseline, a formatting-only edit of it, and a synonym rewording. The judge (deepseek/deepseek-v4-flash-0731) is asked in both orders whether two phrases name the same specialty, and a pair counts as equivalent only when both orders say so. A pair either of whose requests failed (errored or timed out) is excluded. The failures came in bursts of consecutive rows and did not recur on the same rows across versions. The exclusion leaves 1,602, 1,821, and 1,799 of the 2,000 pairs usable for the floor, the formatting edit, and the synonym edit. Under that judge the column’s own floor, two

draws of the same prompt, is 40.1% (Wilson [37.8, 42.6]). The formatting edit flips 42.2% ([40.0, 44.5]) and the synonym edit 49.9% ([47.6, 52.2]).

Pilot calibration. We hand-labeled 201 calibration pairs (one labeler, the author), drawn evenly from the three comparisons and pooled: 150 the judge had called equivalent and 51 it had called different. Each pair was shown in a shuffled order as two phrases with a Same/Different choice, without the judge’s verdict or the comparison it came from. The judge missed 0 true differences among the 150, an upper confidence bound of 2.0% on its miss rate among the pairs it calls equivalent at $\delta_{\text{cal}}=5\%$. The bound is pooled across the three comparisons; each comparison’s 50 labels alone give 5.8%. The judge also called 41 equivalent pairs different. A second model (meta/muse-spark-1.3-contributor) labeled the same pairs and agreed with the hand labels on 74.1% of them; the judge itself agreed on 79.6%. The second model called 20 of the 150 judge-equivalent pairs different, every one of them a pair the author called the same. Under its labels the miss bound would be 18.8%, so the clean side of this calibration rests on the author’s reading.

Pilot outcome and reading. Certifying at $\alpha - \hat{m}$ takes 2.0 percentage points off each budget under the author’s labels: $\alpha=0.2$ becomes 0.180 and $\alpha=0.1$ becomes 0.080. Under the second labeler’s labels the deflation would be 18.8 points, more than the tight budget itself. Stratification is by the cached phrase’s category (8 categories, assigned once from the cache by the same model). In that configuration, both edits are refused on every stratum at both budgets, each run stopping after 360 oracle calls (every stratum at its first look). The composed guarantee holds with probability $1 - \delta - \delta_{\text{cal}}$ under the assumption stated in §8; the judge’s price is the deflation plus the labels. The floor the certifier met is the judge’s, not the model’s. Under the author’s labels, 41 of the 51 pairs the judge called different name the same specialty, across the three comparisons, and 0 of the 150 pairs it called equivalent differ. Weighting the judge’s verdicts by those label shares puts the disagreement of two same-prompt draws, as the author reads them, near 7.9% (band [4.4, 14.5]% from the Wilson intervals of the two shares), inside the loose budget. The two labelers read the judge differently. By the author’s labels it is stricter than the author’s reading and misses nothing. By the second labeler’s labels it is mostly right when it calls a

pair different (42 of 51) and misses 20 of the 150 pairs it calls equivalent. One human labeler and one model cannot say which reading is right; a second human labeler would separate a strict instruction from a lenient labeler. Either way the certifier, which sees only the judge, refused every stratum. The composed guarantee holds for any judge whose miss rate the calibration bounds correctly. What the pilot does not deliver is a judge whose equivalence matches a reader’s, calibrated in both directions.

F The calibration study in detail

The planted study. The study of \$5.6 plants flip rates $p \in [0, 0.25]$ at stratum sizes $\{600, 1,800, 5,000\}$ and budgets $\alpha \in \{0.05, 0.1, 0.2\}$, in both modes, with 2,000 trials per configuration. The worst rate of unsafe certificates, 1.00% (95% CI $[0.65, 1.54]\%$), occurs at $\alpha=0.05$, $p=0.0525$, $n_j=1,800$, in the default mode. The presented-cells rate, which leaves out the sampled flips, exceeds α in at most 0.70% of runs. Of the configurations, 54 are planted just above the budget, at 1.05 to 1.25 times α , where the futility rule often cannot dispose of the stratum at the first look and the sampling fraction climbs. Over these the unsafe rate averages 0.35%. Their certification rate reaches 6.2%, and most of those certificates are correct. At p just above α the realized count of a planted stratum falls at or below the budget in a large share of trials, and the exact bound certifies what is actually there. The reuse-set rate evaluated on default-mode certifications fluctuates past α in frontier configurations. The worst is 100.0% of certified runs at $p=0.0625$, $\alpha=0.05$, certified 0.80% of the time. That is expected, since the default mode does not certify that metric. The strict mode holds its own guarantee within δ_j , with unsafe certificates in at most 0.60% of runs.

The least favorable population. Fix a stratum of N cells of which M flip. Under the pinned schedule the certifier’s decision at each look is a threshold on the flips seen so far. It certifies if the count is at most the largest count whose bound clears the budget (a look that takes the whole stratum cannot certify). It stops for futility if the sample’s rate exceeds the budget, and otherwise it goes on. The flips among each look’s new cells are hypergeometric given the flips already seen. The probability of certifying is therefore a finite sum over the paths of cumulative counts, which the released script evaluates exactly, with the thresholds of the released certifier.

That probability does not increase with M . Take two populations that differ in one cell, a flip in the larger and none in the smaller, and sample both by one shuffle. At every look the larger population’s sample holds at least as many flips. A path of counts that certifies still certifies when every count is lowered: lower counts pass every threshold the path passed and trip no futility stop that it did not. Every shuffle that certifies the larger population therefore certifies the smaller one. In the default mode a certificate is unsafe exactly when $M > \alpha N$, so its probability is largest at $M = \lfloor \alpha N \rfloor + 1$, one flip past the budget. In the strict mode the unsafe event depends on the look, and the script finds the worst M by scanning. Table 18 gives the result for each combination of the grid. The script also cross-checks one combination per budget by running the released certifier on shuffled populations.

Table 18: The exact probability of an unsafe certificate at the least favorable fixed population, for each budget and stratum size of the calibration grid (per-stratum $\delta_j=5\%$; values rounded up).

Budget	default mode, n_j			strict mode, n_j		
	600	1,800	5,000	600	1,800	5,000
$\alpha=0.05$	1.34%	2.27%	1.58%	0.37%	0.87%	0.83%
$\alpha=0.1$	1.90%	2.60%	2.05%	0.88%	0.75%	1.04%
$\alpha=0.2$	2.23%	2.31%	2.62%	0.59%	0.76%	1.02%

Table 19: The default and strict modes on the five edit pairs: certification rate and mean savings over 1,000 replications, in percent. Column *unsafe* is the share of all replications in which the strict mode issues an unsafe certificate, counted per stratum: one for a stratum whose realized flip count exceeds α times the cells it reuses, the event Theorem 3.1 bounds in that mode.

Edit	α	default mode		strict mode		
		cert.	savings	cert.	savings	unsafe
SO formatting-only	0.05	74.4	38.6	4.5	3.4	0.1
SO formatting-only	0.10	98.9	82.6	98.7	80.6	0.0
SO formatting-only	0.20	100.0	88.1	100.0	88.1	0.0
SO synonym rewording	0.05	0.0	0.0	0.0	0.0	0.0
SO synonym rewording	0.10	81.4	55.6	14.2	11.5	0.0
SO synonym rewording	0.20	99.9	87.4	99.9	87.0	0.0
SO scope widening	0.05	1.2	1.0	0.4	0.3	0.4
SO scope widening	0.10	88.8	66.3	37.6	29.5	0.0
SO scope widening	0.20	99.9	86.6	99.9	86.4	0.0
Djinni formatting-only	0.05	0.0	0.0	0.0	0.0	0.0
Djinni formatting-only	0.10	73.8	8.4	0.8	0.2	0.0
Djinni formatting-only	0.20	100.0	52.1	91.1	29.7	0.2
Djinni criteria change	0.05	0.0	0.0	0.0	0.0	0.0
Djinni criteria change	0.10	82.8	14.4	2.7	0.8	0.0
Djinni criteria change	0.20	100.0	48.6	97.6	31.6	0.2

G The strict mode in detail

Table 19 sets the strict mode beside the default mode on the configurations of Table 2, at the three budgets where either certifies. The pattern of \$5.2 holds throughout. Where a stratum’s flip rate is well below the budget the two modes agree, as on the three SO edits at $\alpha=0.2$. One budget step before the default mode’s own frontier the strict mode’s savings fall away: on the synonym and scope-widening edits at $\alpha=0.1$, and on both Djinni edits at $\alpha=0.2$. The strict mode issues unsafe certificates too, at a rate the theorem allows. Counted per stratum, they occur in 4 configurations of the table, most often on SO scope widening at $\alpha=0.05$, in 0.4% of replications. Each rate sits inside its column’s per-stratum level, $\delta_j=0.05$ on the Boolean columns and $\delta_j=0.025$ on the select column.

H The baselines in detail

B1 and B2. On the formatting edit the similarity score’s entire range is 0.96–0.99, because shared row text dominates. The aggregate-only arm runs the same schedule and budgets as the pinned procedure. On the formatting edit it certifies at $\alpha=0.1$ (82.0% savings, 6.46% aggregate error) while the reused cached-TRUE subgroup carries 35.3% error. On the widening edit it

refuses at $\alpha=0.1$ and certifies 95.5% at $\alpha=0.2$ with 44.6% subgroup error. The effect is not an artifact of giving the baseline a shorter schedule.

B2': how the competitor was matched. We matched the competitor to sIVM on every axis we could: the same number of oracle calls, identical labels, identical error target, the same exact bound on the selected population, the same total failure probability δ , and the same reported quantity. Each method's δ is split across the number of tests its own procedure performs: sIVM across the Boolean column's two strata and six looks, $\delta/12$ per test; the competitor across its candidate thresholds, $\delta/3$. The reported quantity is the error among reused cells, not the presented-cells rate sIVM certifies. Run instead on each stratum separately at δ/K with a single look, the competitor would be the non-adaptive special case of sIVM itself.

At $\alpha=0.2$ on the formatting edit the competitor reused 1,910 cells to sIVM's 1,769 from the same 90 labels. At 6.54% realized error it satisfies the bound it was given, against sIVM's 3.28%. On the widening edit the same pattern holds: 1,910 reuses at 9.58% versus 1,756 at 5.47%. The extra volume is drawn from the volatile stratum, 183 cached-TRUE cells on the formatting edit and 196 on the widening edit. With the cached value as the proxy there are only three candidate thresholds, and at $\alpha=0.2$ the aggregate bound clears at the loosest one. The arm therefore selects 100.0% of the rows it was allowed to on the formatting edit and 100.0% on the widening edit. Matched this way, the competitor reduces to B3, reuse everything, with a certificate attached. Its bound is satisfied, its aggregate error is real, and its behavior is indistinguishable from the guarantee-free status quo.

B2': two caveats. First, the cached value is the proxy the reduction names, and the result does not depend on that choice. Run instead with an embedding-interaction proxy, and with its much larger threshold search corrected accordingly, the competitor certifies in 3 of the 4 (edit, budget) configurations and lands in the same place. At $\alpha=0.2$ on the formatting edit it reuses 1,899 cells at 6.53% error, 180 of them cached-TRUE cells that flip at 37.2%. On the widening edit it reuses 1,856 at 9.21%, 168 of them cached-TRUE cells at 50.6%. The proxy changes how much the competitor reuses, not where its errors land. Second, with the cached-value proxy the competitor's per-test level is looser than sIVM's, because it runs fewer tests. The embedding arm, tested at a stricter level than sIVM's, still reuses more than sIVM at $\alpha=0.2$, so that advantage does not rest on the looser level. The comparison is matched on the quantity a user sets (δ in total), not on the level of any individual test.

B3: the threshold sweep. The cells the embedding-transfer baseline reuses at $\tau=0.95$ flip at 6.35% on the formatting edit and at 9.8% on the widening edit, where the population rate is 9.45%. A sweep of τ over the per-row similarities shows the baseline collapsing within a threshold window about 0.03 wide. On the formatting edit it falls from 100% of rows reused at $\tau=0.96$ to 10.7% at 0.98 and none at 0.99. On the widening edit it falls from 99.9% at 0.94 to 8.1% at 0.96 and none at 0.97. The realized error among reused rows

stays at the population flip rate at every operating point (5.6%–6.6% and 8.0%–9.8%). The threshold controls volume, never error, because cross-version similarity carries no per-cell flip signal (B1).

References

- [1] Airtable, Inc. 2026. Using Airtable AI in Fields. Airtable Support, <https://support.airtable.com/articles/8052242094-using-airtable-ai-in-fields>. Accessed 29 September 2026.
- [2] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. 2025. Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control. *Ann. Appl. Stat.* 19, 2 (2025), 1641–1662. <https://doi.org/10.1214/24-AOAS1998> arXiv:2110.01052.
- [3] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avnika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *Proc. VLDB Endow.* 17, 2 (2023), 92–105. <https://doi.org/10.14778/3626292.3626294>
- [4] Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. 2025. Non-Determinism of 'Deterministic' LLM System Settings in Hosted Environments. In *Proc. 5th Workshop on Evaluation and Comparison of NLP Systems*. 135–148. <https://doi.org/10.18653/v1/2025.evalnlp-1.12>
- [5] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proc. 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*. 212–218. <https://doi.org/10.18653/v1/2023.nlposs-1.24>
- [6] Rémi Bardenet and Odalric-Ambrym Maillard. 2015. Concentration Inequalities for Sampling without Replacement. *Bernoulli* 21, 3 (2015), 1361–1385. <https://doi.org/10.3150/14-BEJ605>
- [7] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael I. Jordan. 2021. Distribution-Free, Risk-Controlling Prediction Sets. *J. ACM* 68, 6 (2021), 43:1–43:34. <https://doi.org/10.1145/3478535>
- [8] Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. 2023. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proc. VLDB Endow.* 16, 7 (2023), 1601–1614. <https://doi.org/10.14778/3587136.3587137>
- [9] Uğur Çetintemel, Shu Chen, Alexander W. Lee, Deepti Raghavan, Duo Lu, and Andrew Crotty. 2026. Making Prompts First-Class Citizens for Adaptive LLM Pipelines. In *Proc. Conf. on Innovative Data Systems Research (CIDR)*. arXiv:2508.05012.
- [10] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. How Is ChatGPT's Behavior Changing Over Time? *Harvard Data Sci. Rev.* 6, 2 (2024). <https://doi.org/10.1162/99608f92.5317da47>
- [11] Shu Chen, Junhan Liu, Deepti Raghavan, and Uğur Çetintemel. 2026. VecFlow: Long-Horizon Semantic Processing over Data and Event Streams with LLMs. arXiv:2604.03855
- [12] Shu Chen, Deepti Raghavan, and Uğur Çetintemel. 2025. Continuous Prompts: LLM-Augmented Pipeline Processing over Unstructured Streams. arXiv:2512.03389
- [13] C. J. Clopper and E. S. Pearson. 1934. The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. *Biometrika* 26, 4 (1934), 404–413. <https://doi.org/10.1093/biomet/26.4.404>
- [14] Hanjun Dai, Bethany Yixin Wang, Xingchen Wan, Bo Dai, Sherry Yang, Azade Nova, Pengcheng Yin, Phitchaya Mangpo Phothilimthana, Charles Sutton, and Dale Schuurmans. 2024. UQE: A Query Engine for Unstructured Databases. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2407.09522.
- [15] Tanay Dixit, Daniel Lee, Sally Fang, Sai Sree Harsha, Anirudh Sureshan, Akash Maharaj, and Yunyao Li. 2024. RETAIN: Interactive Tool for Regression Testing Guided LLM Migration. In *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*. 301–310. <https://doi.org/10.18653/v1/2024.emnlp-demo.31> arXiv:2409.03928.
- [16] H. F. Dodge and H. G. Romig. 1929. A Method of Sampling Inspection. *Bell Syst. Tech. J.* 8, 4 (1929), 613–631. <https://doi.org/10.1002/j.1538-7305.1929.tb01240.x>
- [17] Nazarii Drushchak and Mariana Romanynshyn. 2024. Introducing the Djinni Recruitment Dataset: A Corpus of Anonymized CVs and Job Postings. In *Proc. 3rd Ukrainian Natural Language Processing Workshop (UNLP) at LREC-COLING 2024*. 8–13. <https://doi.org/10.63317/2dcvy45ws6yb>
- [18] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks. *Proc. Natl. Acad. Sci. USA* 120, 30 (2023), e2305016120. <https://doi.org/10.1073/pnas.2305016120>
- [19] Google LLC. 2026. Use the AI Function in Google Sheets. Google Docs Editors Help, <https://support.google.com/docs/answer/15877199>. Accessed 29 September 2026.
- [20] Ashish Gupta, Inderpal Singh Mumick, and Kenneth A. Ross. 1995. Adapting Materialized Views after Redefinitions. In *Proc. ACM SIGMOD Int. Conf. on Management of Data*. 211–222. <https://doi.org/10.1145/223784.223817>

- [21] Horace He and Thinking Machines Lab. 2025. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab: Connectionism, technical report. <https://doi.org/10.64434/tml.20250910> Accessed 15 August 2026.
- [22] Joseph M. Hellerstein, Peter J. Haas, and Helen J. Wang. 1997. Online Aggregation. In *Proc. ACM SIGMOD Int. Conf. on Management of Data*. 171–182. <https://doi.org/10.1145/253260.253291>
- [23] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30. <https://doi.org/10.1080/01621459.1963.10500830>
- [24] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. 2021. Time-Uniform, Nonparametric, Nonasymptotic Confidence Sequences. *Ann. Statist.* 49, 2 (2021), 1055–1080. <https://doi.org/10.1214/20-AOS1991>
- [25] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proc. ACM Manag. Data* 2, 3 (2024), 1–26. <https://doi.org/10.1145/3654989>
- [26] Daniel Kang, Edward Gan, Peter Bailis, Tatsunori Hashimoto, and Matei Zaharia. 2020. Approximate Selection with Guarantees Using Proxies. *Proc. VLDB Endow.* 13, 12 (2020), 1990–2003. <https://doi.org/10.14778/3407790.3407804>
- [27] Daniel Kang, John Guibas, Peter Bailis, Tatsunori Hashimoto, Yi Sun, and Matei Zaharia. 2021. Accelerating Approximate Aggregation Queries with Expensive Predicates. *Proc. VLDB Endow.* 14, 11 (2021), 2341–2354. <https://doi.org/10.14778/3476249.3476285>
- [28] Sanjay Krishnan, Jiannan Wang, Michael J. Franklin, Ken Goldberg, and Tim Kraska. 2015. Stale View Cleaning: Getting Fresh Answers from Stale Materialized Views. *Proc. VLDB Endow.* 8, 12 (2015), 1370–1381. <https://doi.org/10.14778/2824032.2824037>
- [29] K. K. Gordon Lan and David L. DeMets. 1983. Discrete Sequential Boundaries for Clinical Trials. *Biometrika* 70, 3 (1983), 659–663. <https://doi.org/10.1093/biomet/70.3.659>
- [30] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2026. SemBench: A Benchmark for Semantic Query Processing Engines. *Proc. VLDB Endow.* 19, 8 (2026), 1754–1767. <https://doi.org/10.14778/3811243.3811249>
- [31] Alexander W. Lee, Benjamin Han, Shayak Sen, Sam Yeom, Uğur Çetintemel, and Anupam Datta. 2026. Evergreen: Efficient Claim Verification for Semantic Aggregates. arXiv:2604.26180
- [32] Mark Lindeman and Philip B. Stark. 2012. A Gentle Introduction to Risk-Limiting Audits. *IEEE Secur. Privacy* 10, 5 (2012), 42–49. <https://doi.org/10.1109/MSP.2012.56>
- [33] Paweł Liskowski and Kyle Schmaus. 2026. Streaming Model Cascades for Semantic SQL. arXiv:2604.00660
- [34] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. 2025. Palimpsest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *Proc. Conf. on Innovative Data Systems Research (CIDR)*. <https://vldb.org/cidrdb/papers/2025/p12-liu.pdf>, accessed 30 September 2026.
- [35] Muhammad Mansoor, Tahir Ahmad, and Yeo-Chan Yoon. 2026. Risk-Constrained Freshness-Aware Semantic Caching for Open-Web Retrieval-Augmented LLMs. arXiv:2607.04281
- [36] Denis Mayr Lima Martins and Gottfried Vossen. 2026. Prompt as a Data Type: In-Database LLM Prompt Management and Rewriting. arXiv:2607.21756
- [37] Andreas Maurer and Massimiliano Pontil. 2009. Empirical Bernstein Bounds and Sample Variance Penalization. In *Proc. Conf. on Learning Theory (COLT)*. Online proceedings, paper 012; also arXiv:0907.3740.
- [38] Microsoft Corporation. 2026. COPILOT Function. Microsoft Support, <https://support.microsoft.com/en-us/excel/functions/copilot-function>. Preview feature, withdrawn 14 September 2026; accessed 29 September 2026.
- [39] Notion Labs, Inc. 2026. Notion AI Autofill for Databases. Notion Help Center, <https://www.notion.com/help/autofill>. Accessed 29 September 2026.
- [40] Azam Nouri. 2026. StepCache: Step-Level Reuse with Lightweight Verification and Selective Patching for LLM Serving. arXiv:2603.28795
- [41] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2025. An Empirical Study of the Non-Determinism of ChatGPT in Code Generation. *ACM Trans. Softw. Eng. Methodol.* 34, 2 (2025), 42:1–42:28. <https://doi.org/10.1145/3697010> arXiv:2308.02828
- [42] Nicholas Pangakis, Samuel Wolken, and Neil Fasching. 2023. Automated Annotation with Generative AI Requires Validation. arXiv:2306.00176
- [43] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proc. VLDB Endow.* 18, 11 (2025), 4171–4184. <https://doi.org/10.14778/3749646.3749685>
- [44] S. J. Pocock. 1977. Group Sequential Methods in the Design and Analysis of Clinical Trials. *Biometrika* 64, 2 (1977), 191–199. <https://doi.org/10.1093/biomet/64.2.191>
- [45] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. 2026. vCache: Verified Semantic Prompt Caching. In *Proc. Int. Conf. on Learning Representations (ICLR)*. arXiv:2502.03771.
- [46] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting. In *Proc. Int. Conf. on Learning Representations (ICLR)*. arXiv:2310.11324.
- [47] R. J. Serfling. 1974. Probability Inequalities for the Sum in Sampling without Replacement. *Ann. Statist.* 2, 1 (1974), 39–48. <https://doi.org/10.1214/aos/1176342611>
- [48] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proc. VLDB Endow.* 18, 9 (2025), 3035–3048. <https://doi.org/10.14778/3746405.3746426>
- [49] Shreya Shankar, Haotian Li, Parth Asawa, Madelon Hulsebos, Yiming Lin, J. D. Zambrescu-Pereira, Harrison Chase, Will Fu-Hinthorn, Aditya G. Parameswaran, and Eugene Wu. 2024. SPADE: Synthesizing Data Quality Assertions for Large Language Model Pipelines. *Proc. VLDB Endow.* 17, 12 (2024), 4173–4186. <https://doi.org/10.14778/3685800.3685835>
- [50] Shreya Shankar, Sepanta Zeighami, and Aditya Parameswaran. 2026. Task Cascades for Efficient Unstructured Data Processing. *Proc. ACM Manag. Data* 4, 1 (2026), 1–26. <https://doi.org/10.1145/3786702> SIGMOD 2026.
- [51] Snowflake Inc. 2026. Snowflake Cortex AI Functions: AI_COMPLETE and AI_CLASSIFY. Snowflake Documentation, https://docs.snowflake.com/en/sql-reference/functions/ai_complete. Accessed 29 September 2026.
- [52] Stack Overflow. 2023. Stack Overflow Annual Developer Survey 2023. <https://survey.stackoverflow.co/2023/>. Accessed 1 October 2026.
- [53] Petter Törnberg. 2023. ChatGPT-4 Outperforms Experts and Crowd Workers in Annotating Political Twitter Messages with Zero-Shot Learning. arXiv:2304.06588
- [54] Ruodu Wang and Aaditya Ramdas. 2022. False Discovery Rate Control with E-values. *J. R. Stat. Soc. Series B* 84, 3 (2022), 822–852. <https://doi.org/10.1111/rssb.12489>
- [55] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proc. Int. Conf. on Learning Representations (ICLR)*. arXiv:2203.11171.
- [56] Ian Waudby-Smith and Aaditya Ramdas. 2024. Estimating Means of Bounded Random Variables by Betting. *J. R. Stat. Soc. Series B* 86, 1 (2024), 1–27. <https://doi.org/10.1093/jrsssb/qkad009>
- [57] Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. 2025. Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2506.09501.
- [58] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2025. Cut Costs, Not Accuracy: LLM-Powered Data Processing with Guarantees. *Proc. ACM Manag. Data* 3, 6 (2025), 1–26. <https://doi.org/10.1145/3769776> SIGMOD 2026.
- [59] Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can Large Language Models Transform Computational Social Science? *Comput. Linguist.* 50, 1 (2024), 237–291. https://doi.org/10.1162/coli_a_00502